Boltzmann Brain: A Versioned, Distributable, and Model-Agnostic Knowledge Architecture

Alex Fiorenza
alexfiorenza2012@gmail.com
Gaussia

github.com/gaussia-labs/papers
gaussia.ai/papers
huggingface.co/gaussia-labs

Abstract

Knowledge produced with large language models (LLMs) tends to become locked inside a single provider, agent, or vector store, where it is hard to audit, reindex, version, or move. We propose **Boltzmann Brain**, a portable knowledge architecture built on one principle: the brain conserves, validates, and retrieves knowledge, while an external and interchangeable LLM processes, contextualizes, and uses it. The brain embeds no model of its own. Knowledge is stored as typed, content-addressed blocks organized into five memory modules: canonical, episodic, semantic, procedural, and provenance. Each module bundles its blocks, a Merkle DAG that pins the exact composition of a version, and its indices. We separate three levels of hashes, for block identity, module composition, and transportable file, and package each module as a separate OCI Artifact, which yields selective installation and incremental updates. Interaction happens through the *Boltzmann Protocol*: the LLM proposes candidate blocks and consumes Evidence Bundles, while the protocol governs what is committed and returns evidence, never prose. Canonical blocks are the root of evidence, recording what was observed, and every derived interpretation cites them through provenance. Wrong knowledge must be removable without punching holes in the current snapshot, so non-episodic modules drop blocks, rebuild their Merkle DAGs, and cascade through provenance, with redaction reserved for destroying retained bytes. This version fixes the interoperability surface, from the block envelope to a conformance kit of golden vectors, and separates integrity from *authenticity*, which hashing alone cannot supply. One detached signature over a snapshot covers every module root and the entire prior history, the authorized keys travel inside the brain, and validity is decided by position in the hash-linked chain, never by a forgeable timestamp. Divergence becomes reconciliation: snapshots name several parents, compositions merge as sets of immutable blocks, and the result is re-validated as an ingestion. We close with the adversarial model this design implies, including what it leaves open.

Keywords: knowledge representation, retrieval-augmented generation, content addressing, Merkle DAG, OCI Artifacts, memory systems, LLM agents

1 Introduction

Large language models have made it cheap to *produce* knowledge (notes, summaries, extracted facts, procedures), but expensive to *own* it. In practice, the knowledge a user accumulates while working with an LLM tends to be captured in a form dictated by whichever provider, agent framework, or vector database happens to be in use. Embeddings are computed with a specific

model, chunks are shaped by a specific pipeline, and the resulting store is rarely portable, verifiable, or re-interpretable. When the model changes, the provider changes, or the indexing strategy changes, the knowledge is often effectively lost, or must be rebuilt from scratch.

This paper argues that knowledge should be a *first-class, portable artifact* that outlives any particular model or vendor. We present **Boltzmann Brain**, an architecture whose sole responsibility is to conserve information, organize it into memories with distinct purposes, maintain verifiable versions, and retrieve relevant evidence. The brain contains no language model of its own. It integrates with whatever LLM the user chooses, which interprets sources during ingestion and contextualizes results during query. The design is captured by a single guiding idea:

> *The brain conserves, validates, and retrieves knowledge. An external LLM processes, contextualizes, and uses it.*

The responsibility boundary. The brain does not write the final answer. It returns knowledge blocks and evidence; an agent, interface, or downstream application decides how to contextualize them (Figure 1). This boundary is what keeps the architecture independent of any provider: the Boltzmann Protocol governs structure, identity, validation, and distribution, while the semantic work (reading, extracting concepts, recognizing episodes, proposing procedures, and phrasing answers) is delegated to an interchangeable external model.

Contributions. This work makes the following contributions:

1. A set of design principles and problems (Section 2) for portable, verifiable, model-agnostic knowledge, and an argument for why six requirements that existing work satisfies separately must hold over a single artifact (Section 4).
2. A mapping from biological memory principles and classic systems structures to concrete architectural decisions (Section 3).
3. A five-module memory model (canonical, episodic, semantic, procedural, and provenance; Section 5) that treats the canonical module as the root of observed evidence, together with an internal module anatomy of content-addressed blocks, Merkle DAGs, and indices (Section 6).
4. A distribution model based on OCI Artifacts that enables selective installation and incremental updates (Section 7), and a model-agnostic protocol with explicit ingestion and query contracts (Sections 9–11).
5. A treatment of how knowledge leaves a brain (Section 12): drop with Merkle rebuild and provenance cascade for non-episodic modules, a privileged cascade when canonical evidence is excluded, supersession for episodic memory, pruning of unreachable blocks, and redaction when retained bytes must be destroyed.
6. A normative encoding for all of the above: the block envelope and its canonical serialization (Section 6.2), the Merkle construction and its layout identifier (Section 6.3), the composition and snapshot documents (Sections 6.4 and 6.5), the schema-versioning rules that let old clients read new brains (Section 6.9), and a conformance kit of golden vectors (Section 16).
7. A specification of authenticity (Section 8): signatures over snapshots, a trust root that travels inside the brain, signing scopes derived from the protocol's own operations, quorum-governed rotation of the key list, and revocation expressed by position in the hash-linked chain rather than by a clock.

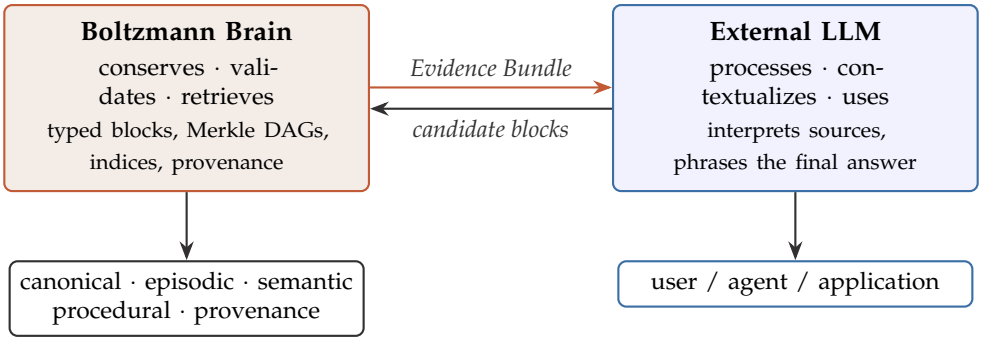

Figure 1: Separation of responsibilities. The brain owns knowledge, identity, and verification; the external LLM owns interpretation and phrasing. The two communicate through a stable contract: the LLM sends candidate blocks during ingestion and receives Evidence Bundles during query.

8. A reconciliation model for diverged brains (Section 13): multi-parent snapshots, a three-way merge over sets of immutable blocks, merge, rebase, and squash recast as recording strategies that differ in attribution rather than in outcome, and a rule that resolves the conflicts that remain by re-validating the result as an ingestion.
9. A Security Considerations section (Section 15) stating what the protocol guarantees, what it does not, and where the burden sits, including the structural exposure created by routing preserved evidence into an external model.

Scope of this version. This document fixes both an architecture and the interoperability surface that follows from it. Everything two independent implementations must agree on to exchange a brain is normative here: the block envelope, the canonical serialization, the Merkle construction, the composition and snapshot documents, the distribution encoding, and the ingestion and query contracts (Section 16 states how an implementation demonstrates agreement). What stays deliberately open is what an implementation may choose without breaking exchange: consolidation and contradiction-detection strategies, ranking and fusion, index engines, and pack sizing. The two questions an earlier version left open in the normative surface itself, authenticity of authorship and reconciliation of diverged brains, are specified here (Sections 8 and 13). Section 15 states the exposures that remain, and Section 17 states what the protocol still owes, chiefly freshness, which no signature establishes.

Notational conventions. The key words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** are used as defined in RFC 2119 [4]. A *conforming implementation* is one that satisfies every MUST in this document for the roles it claims (Section 16). Documents are shown as JSON for readability; the bytes that are hashed are always the canonical serialization of Section 6.2, never the pretty-printed form.

2 Design Principles and Problems

2.1 Problems this architecture addresses

Boltzmann Brain is designed to address a cluster of related failures in how LLM-era knowledge is currently stored and consumed:

- Preventing knowledge from being trapped inside a particular provider, agent, or vector database.

- Maintaining a source of evidence that allows knowledge to be reindexed and reinterpreted as models and methods evolve.
- Distinguishing concrete experiences, general facts, and procedures.
- Updating knowledge without losing history, provenance, or reproducibility.
- Removing knowledge that is obsolete, costly, or legally untenable, without silently compromising verifiability.
- Distributing a complete brain or only selected modules.
- Allowing interchangeable external LLMs to query and contribute knowledge through a stable contract.

2.2 Guiding principles

Ten principles follow from these problems and constrain every subsequent design decision:

1. Knowledge lives in blocks; indices are not the source of truth.
2. Original evidence is preserved by default; interpretations may change. Dropping a canonical block is allowed only under explicit policy, and it forfeits re-derivation from that source.
3. Every version must be identifiable, verifiable, and reconstructible.
4. Modules must be distributable and installable selectively.
5. The protocol must not embed an LLM of its own nor depend on a specific provider.
6. Specialized structures compose; no single index answers every query.
7. Queries are declarative and index-agnostic: callers ask for knowledge, and the selection and combination of indices is left to the implementation.
8. Forgetting is explicit and auditable: non-episodic modules support drop with Merkle rebuild and provenance cascade; dropping canonical evidence is privileged and always cascades; content may be redacted under policy, but never silently.
9. Integrity and authenticity are separate claims. Integrity is verifiable by anyone, offline, with no configuration; authorship is a signed assertion evaluated against a trust root that travels inside the brain; and neither claim may ever be reported as the other.
10. Divergence is reconciled, not overwritten. Two brains that advanced from a common ancestor are merged as sets of immutable blocks, and the result is re-validated before it is committed.

These principles are intentionally conservative. They privilege durability and auditability over convenience, and they treat the LLM as a powerful but replaceable interpreter rather than as the system of record.

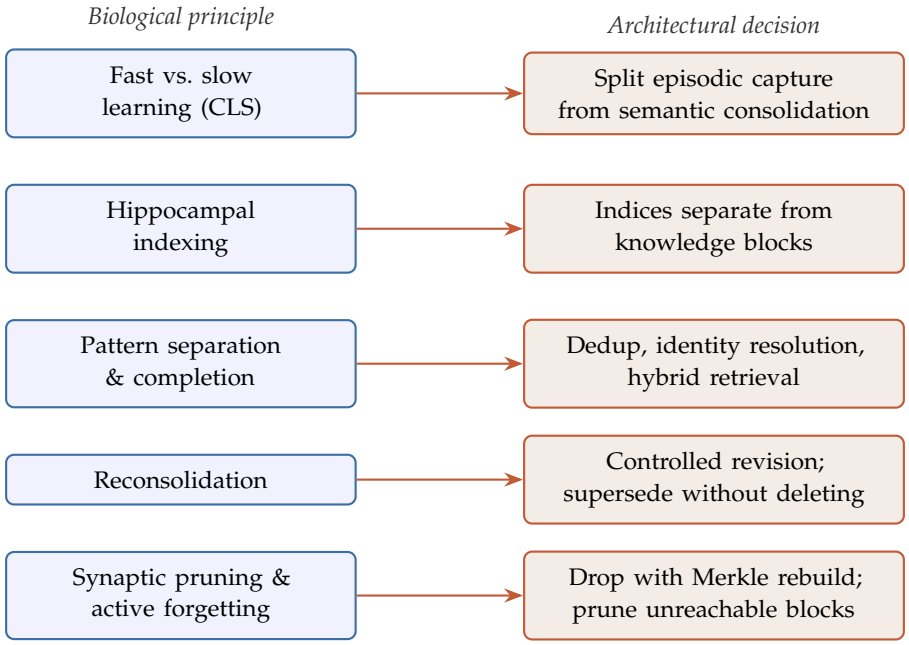

Figure 2: Translation of biological memory principles into architectural decisions. The mapping is by analogy, not by literal correspondence between brain regions and system components.

3 Inspirations

3.1 Biology of memory

The architecture takes *general principles* from neuroscience, not literal equivalences between brain regions and databases. The scientific literature describes multiple memory systems and an interaction among fast learning, stable knowledge, consolidation, associative recall, and adaptive forgetting [10, 15, 16, 23, 32, 33, 35, 42]. Figure 2 summarizes the translation from biological principles to architectural decisions.

Complementary learning systems (CLS) theory distinguishes a fast, hippocampus-associated mechanism from slower, integrative cortical learning [16]. For Boltzmann Brain this suggests separating the *capture of episodes* from *semantic consolidation*. The hippocampal indexing theory further motivates a distinction between the index that reactivates a memory and the distributed content of that memory [10], a distinction we mirror in our separation of indices from knowledge blocks. Pattern separation inspires deduplication, identity resolution, and contradiction detection; pattern completion inspires recovering a full memory from a partial query through hybrid search [15, 32, 42]. Finally, reconsolidation suggests that retrieving knowledge can open a controlled revision [23, 33], while work on pruning and active forgetting suggests that removing what is not reinforced is a regulated process rather than a failure, and that accessibility can be reduced without destroying the underlying evidence [8, 30, 35]. Section 12 develops this last point into concrete retention and erasure mechanisms.

3.2 Systems structures

Alongside biology, the architecture reuses well-understood systems structures. Table 1 lists each one and its contribution.

Table 1: Systems structures and their contribution to Boltzmann Brain.

Concept	Contribution to Boltzmann Brain
Content addressing	The identity of a block derives from its content.
Merkle DAG	A root identifies the exact composition of a module.
B^+ tree	Exact lookup, hierarchies, and ordered range scans.
Inverted index	Lexical search over words and terms.
Vector index	Approximate retrieval by semantic similarity.
Graph	Explicit relations, provenance, associative expansion.
Mark-and-sweep collection	Reclaiming blocks unreachable from a retained root.
OCI Artifact	Interoperable packaging and distribution of modules.
Skill / MCP / CLI / API	Contract for an external LLM to query and ingest.

4 Related Work and the Case for a New Protocol

4.1 What already exists

The industry is already solving pieces of this problem, but no single artifact yet integrates a canonical source, typed memories, fine-grained versioning, portable indices, modular distribution, and a model-agnostic access protocol. Table 2 positions the most relevant efforts. We then discuss them in turn.

Table 2: Related efforts and their distance from Boltzmann Brain.

Project / standard	What it contributes	Distance from Boltzmann Brain
Open Knowledge Format	Markdown + YAML as a portable, human-readable format for agents.	Formalizes canonical exchange, but defines no access protocol, biological memory model, indices, or compiled distribution.
GBrain	Git repository as a durable registry; hybrid search, graph, and agent access.	Reinforces the independence of knowledge base from agent; its primary distribution follows the Git/repository model.
Microsoft GraphRAG	Extracts entities, relations, communities, and summaries; combines graph and text.	An indexing and retrieval pipeline, not a distributable format for a complete brain.
Cognee	Turns data into chunks, embeddings, nodes, and relations; combines vector and graph.	Provides memory operations and enrichment, but does not define the proposed OCI/Merkle bundle.

The Open Knowledge Format (OKF) formalizes the knowledge-wiki pattern through Markdown and YAML, emphasizing independence between producers and consumers [11]. GBrain separates durable knowledge from agent behavior and offers interchangeable engines and agent integration [38–40]. GraphRAG and Cognee illustrate the convergence toward hybrid representations that mix text, vectors, and graphs [6, 7, 18, 19].

Substrate rather than alternatives. Two standards that recur throughout this paper are deliberately absent from the comparison. MCP standardizes how applications connect to external resources and tools [1], and the OCI ecosystem provides content-addressed manifests, descriptors, and blobs whose structure is itself an external Merkle DAG [25, 26, 29]. Neither is an alternative to a knowledge architecture, because one standardizes a call and the other a transport, and this design adopts both instead of competing with them (Sections 7 and 9). They return in Section 4.3, where the question is what a stack assembled out of them still fails to provide.

4.2 Six requirements that must hold simultaneously

Proposing a new protocol requires more than observing that existing tools are imperfect. The argument of this paper is that the problems of Section 2 translate into six requirements which, to be useful, must hold *at the same time and over the same artifact*:

R1. Canonical evidence. The original source is preserved and addressable, so that any interpretation can be audited against it and re-derived when methods improve. Without this, knowledge cannot be re-interpreted, only trusted.

R2. Typed memories. Concrete experiences, consolidated facts, and procedures are distinguished rather than collapsed into undifferentiated chunks. Without this, "what happened in the class of May 14" and "the definition of a Fourier series" compete in a single similarity ranking.

R3. Verifiable versioning at knowledge granularity. Versions are identifiable, verifiable, and reconstructible at the granularity of a unit of knowledge rather than a file. Without this, a corrected fact cannot be distinguished from an edited document.

R4. Derived, replaceable indices. Indices are views that can be rebuilt or replaced when an engine or embedding model changes. Without this, knowledge dies with the model that indexed it.

R5. Selective and incremental distribution. A consumer can install one module and update only what changed. Without this, sharing knowledge means shipping everything, every time.

R6. Model-agnostic access contract. A stable, declarative contract lets interchangeable models query and contribute. Without this, the store is reachable only through whoever wrote the integration.

Table 3 scores the efforts of Table 2 against these six axes. The pattern is not that existing work is weak; it is that each effort is strong on a different axis. Portable formats concentrate on R1 and leave retrieval undefined, while retrieval pipelines concentrate on R4 and treat versioning and distribution as someone else's problem. The closest neighbor, GBrain, is partial on almost every axis precisely because it inherits its versioning and distribution from Git rather than defining them at the knowledge layer. What no row does is combine more than a fraction of the six, which is the gap this paper addresses.

4.3 Why composition is not enough

Every one of the six requirements is met somewhere in the wider ecosystem, once the standards of the previous subsection are counted, so the obvious objection is that the answer is integration rather than a new protocol: put the sources in Git, run GraphRAG over them, expose the result through MCP, and ship it with OCI. That stack is worth building, and much of it is what a

Table 3: Design coverage of the six requirements of Section 4.2: ● full, ◐ partial, ○ absent. The last row states the design intent of this version, not a validated implementation.

Project / standard	R1	R2	R3	R4	R5	R6
Open Knowledge Format	●	◐	◐	○	○	○
GBrain	●	◐	◐	◐	◐	◐
Microsoft GraphRAG	◐	◐	○	◐	○	○
Cognee	◐	◐	○	◐	○	◐
Vector store + RAG pipeline	○	○	○	○	○	○
Boltzmann Brain (this work)	●	●	●	●	●	●

conforming implementation would look like. What it does not produce is a portable artifact, for four reasons.

Identity does not compose. Each layer carries its own notion of what a thing is: a commit, a pipeline output, a registry digest. None of them binds a retrieved item to the source it came from and to the snapshot it belongs to. That binding is exactly what an Evidence Bundle asserts when it reports `verified`, and it is why this architecture separates three levels of hashes (Section 6) instead of borrowing one. Verifiability is a property of the whole path from source to answer, so it cannot be added by a layer that only sees its own inputs.

Granularity mismatch. Version control versions files; knowledge changes at the granularity of a claim. A corrected definition, its evidence, and the procedures that depended on it are not a diff over lines, and recovering that structure from a file history means reconstructing after the fact what the ingestion step already knew. R3 asks for versioning where the knowledge is, which is the block.

The layering runs the wrong way. In a retrieval pipeline the index is the product: documents are consumed to build embeddings and graphs, and those derived structures become the thing that is queried, stored, and eventually migrated. R4 requires the opposite ordering, with blocks as the durable artifact and indices as disposable views over them. Inverting an existing pipeline is not a matter of configuration, because a pipeline whose output is its state has no representation of the knowledge that survives it.

Access and transport standardize the edges, not the middle. MCP standardizes the call and OCI standardizes the bytes, which is why both appear in this design rather than being replaced by it. Neither says what is on the other side of the call or inside the blob. Two MCP servers over the same corpus share no verifiable state, and two registries holding the same brain agree on digests while knowing nothing about modules, memory types, or snapshots. The missing piece sits between them: a format for typed, content-addressed blocks plus a contract for reading and extending them.

Why now. The same argument would have been premature a few years ago, when the model, the pipeline, and the application were usually chosen once. Today embedding models are deprecated on a timescale of months and agent frameworks turn over faster still, so the useful life of accumulated knowledge routinely exceeds the life of the tools that produced it. The container ecosystem went through this transition: engines and registries became interchangeable only once image identity and packaging were specified independently of any implementation.

LLM knowledge is at the earlier stage, with capable engines and no shared artifact, and this paper argues for specifying the artifact.

Complementarity. Boltzmann Brain is therefore complementary rather than competing. It can consume OKF-style canonical sources, borrow hybrid retrieval ideas from GraphRAG and Cognee, expose an MCP interface, and ride on OCI for distribution, while contributing the missing piece: a versioned, typed, model-agnostic *bundle* whose modules can be installed and updated selectively, and whose contents remain verifiable no matter which model reads them.

5 Knowledge Modules

The brain's OCI Artifact can contain five logical modules. Each module is installable as an independent physical unit and contains blocks, a Merkle DAG, and the indices needed for query. Table 4 names each module by the question it answers.

Table 4: The five memory modules, illustrated with a university brain.

Module	Question it answers	Examples
Canonical	What material was observed?	Notes, books, PDFs, lectures, code, datasets.
Episodic	What happened in a concrete context?	Class of May 14, an error in an exercise, a professor's correction.
Semantic	What general knowledge was consolidated?	Definitions, formulas, facts, relations.
Procedural	How is a task performed?	Solving an integral, computing Fourier coefficients.
Provenance	Where did it come from and how was it transformed?	Page, source, pipeline, model, validator, date.

Canonical module. Preserves observed evidence, not truth claims. A canonical block asserts that a source was incorporated and that it *contains* certain bytes; it does not declare those bytes correct. Because every semantic and procedural interpretation cites canonical evidence through provenance, this module is the root of re-derivation: without the source, derived knowledge cannot be rebuilt cleanly from below.

The module holds two content-addressed layers. The *original* is the blob as observed (PDF, repository archive, conversation export, image, audio, and so on). An optional *normalized view* is a deterministic transform of that blob (plain text, Markdown, or a structured extract) produced by a named pipeline whose identity is recorded in provenance. Normalized views live in canonical rather than semantic memory because they are still evidence for ingestion, not consolidated knowledge. Both layers are addressed by hash, so re-registering an identical original is a no-op.

A canonical block therefore carries only what its bytes determine: its `block_id`, media type, size, and optionally the digest of its normalized view. It carries nothing else, because its identity must depend on the observed bytes alone for re-registering an identical original to be a no-op: if the registering actor or the moment of registration were part of the hashed content, two people incorporating the same source would obtain two different blocks and deduplication would never trigger. Everything historical or actor-dependent is recorded in the provenance module instead, which is where relations that depend on who acted and when already live: the license or retention policy under which the source is held, the actor who incorporated it, the timestamp,

and the supersession edge toward an earlier edition when a new one replaces it (Sections 10 and 12).

Episodic module. Records events and experiences with time, context, participants, outcome, and evidence. It is the only module that stays append-only: corrections generate new episodes or supersession relations rather than drops that rewrite the past (Section 12).

Semantic module. Holds consolidated general knowledge: concepts, formulas, facts, relations, and constraints. A Fourier-series formula belongs primarily here, linked to its canonical sources.

Procedural module. Represents action sequences, conditions, decisions, alternative paths, and success criteria, for example how to obtain the Fourier coefficients of a given function.

Provenance module. Connects derived blocks to evidence, transformations, and tool versions. It is key to answering what must be recomputed when a source changes, and it is the ledger of removals: every drop, cascade, and redaction is recorded there so that exclusion stays auditable (Section 12). The actor named in a provenance record is a *declared* identifier; what turns it into an authenticated identity is the signature over the snapshot that committed the record (Section 8), which is why authenticity belongs in this specification.

Provenance is not free-form annotation. It carries six kinds of record (Table 5), and the set is closed because every other part of the protocol depends on being able to ask a precise question of it: which blocks cite this evidence, what produced this block, and what was removed by whom.

Table 5: The provenance records, and what each one makes answerable.

Record	What it states
Registration	A source was incorporated: by which actor, when, from where, and under what license or retention policy.
Normalization	A normalized view was produced from an original by a named deterministic pipeline.
Derivation	A block was derived from cited evidence by a named producer, within a named task.
Supersession	A block takes precedence over an earlier one.
Demotion	A block's retrieval priority was reduced, without removing it.
Removal	A block was excluded, pruned, or redacted: by which mechanism, actor, and reason.

Two distinctions inside these records carry weight elsewhere. An *actor* is who performed an operation, while a *producer* is what generated a derived block, recorded at the granularity a batch invalidation needs: a model, a pipeline, an ingestion batch, or an actor. That granularity is what makes Section 12.3's batch invalidation expressible as a query over provenance. And because registration records live here rather than inside canonical blocks, re-registering an identical source deduplicates while still recording that a second actor incorporated it.

6 Anatomy of a Module

A module carries three responsibilities that coexist in the same physical unit (Figure 3): *blocks* contain knowledge; the *Merkle DAG* defines which blocks form a version; *indices* make it possible to find blocks without scanning all of them.

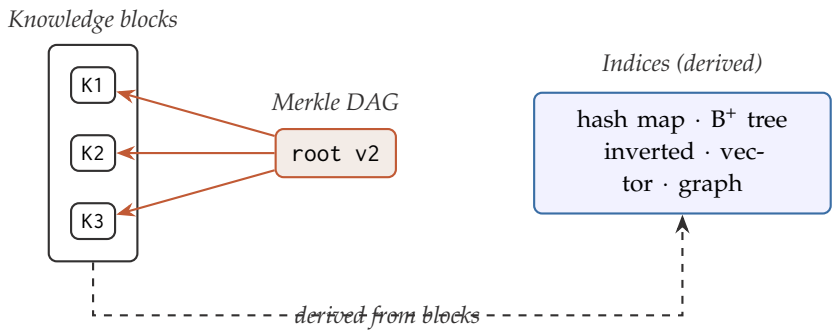

Figure 3: Blocks, a Merkle DAG, and indices coexist within the same module. The Merkle root pins the exact set of blocks that make up a version; indices are derived and can be rebuilt from the blocks.

6.1 Knowledge blocks

A block is a logical unit of knowledge with a deterministic serialization. It may be an episode, a concept, a fact, a relation, a procedure, or a canonical fragment. Its identifier is computed from its content:

$$\texttt{block\_id} = \text{SHA-256}\big(\texttt{canonical\_serialization(envelope)}\big). \tag{1}$$

If the content changes, a new block is born. The previous one may be kept for history, audit, or rollback.

The envelope. What is hashed is not the payload alone but an *envelope* that carries exactly five keys:

```
{
  "boltzmann":      1,
  "memory_type":    "semantic",
  "payload":        { ... },
  "schema_version": 1,
  "serialization":  "jcs/1"
}
```

A conforming implementation MUST reject an envelope carrying any other key, or missing any of these. Binding the memory type and the schema version into the hash is what makes identity self-describing: the same payload read as an episode and as a semantic fact is two different blocks, and stored bytes can be decoded without consulting anything outside them.

Everything derived, physical, or actor-dependent stays outside the envelope: the storage location, the index state, and the moment of registration are not part of what a block *is*. This is the same argument that keeps canonical blocks free of registration metadata (Section 5), applied uniformly.

Two conventions that keep identity unambiguous. An absent optional field MUST be omitted from the payload rather than serialized as `null`, so that `{"a":1}` and `{"a":1,"b":null}` are the same block rather than two blocks with the same meaning. Blocks MUST be treated as frozen: correcting a block means creating a new one.

Content a block names but does not carry. A payload is JSON, so a block whose datum is large or binary (a PDF, an audio file, a repository archive) names it by digest and leaves the bytes in the store. A block therefore has a set of *content digests*, possibly empty. Every operation

that must account for those bytes, namely packing a layer, marking reachability before a prune, and destroying bytes on redaction, MUST consult that set rather than inspect the block's type, so that a schema which starts naming content is handled correctly by all of them at once.

Decoding. A conforming implementation MUST verify, when decoding stored bytes, that re-serializing the decoded block reproduces those bytes exactly. Bytes that are not already in canonical form would hash to a different `block_id` than the one under which they are stored, so they MUST be rejected rather than silently normalized.

6.2 Canonical serialization

Two clients that disagree on serialization compute different identifiers for the same knowledge, and then deduplication never triggers, structural sharing stops working, and two parties holding identical knowledge cannot tell that they do. The serialization is therefore normative, and it is named in every envelope so that a future one can coexist with it rather than replace it.

This version uses **JCS**, the JSON Canonicalization Scheme of RFC 8785 [34]: object keys sorted by UTF-16 code point, no insignificant whitespace, and a fixed number format. Its identifier is `jcs/1`. JCS was chosen over a binary encoding because a block is a small structured record and the protocol is meant to be implemented in several languages: a canonical form that a human can read and grep is worth more here than compactness, and RFC 8785 has independent implementations across the language ecosystems a client is likely to be written in.

Two value domains are excluded from a payload

JCS defines float serialization through ECMAScript rules that are subtly hard to reproduce identically across languages, and integers outside the IEEE-754 safe range ($\pm(2^{53}-1)$) lose precision in any JSON parser backed by doubles. Either divergence would mean two conforming clients computing different `block_id` values for the same knowledge. Both MUST therefore be rejected when a block is constructed, rather than at hashing time. Represent a decimal as a string, or as an integer scaled by a documented factor.

6.3 Merkle DAG

On their own, blocks are just a bag of content-addressed units: each is addressable by its hash, but nothing names, verifies, or diffs a whole *version* of the module. The Merkle DAG solves exactly this. It stores no meaning by itself; it maintains references by hash and produces a single root that commits to the exact set and structure of blocks in a version. That root *is* the identity of the version, and it is what makes Principle 3 (identifiable, verifiable, and reconstructible versions) concrete.

How it works. The leaves of the DAG are `block_ids` (content hashes); each internal node hashes the hashes of its children; the root hashes down to everything below it. Changing any block changes its hash, which changes every ancestor up to the root. Three operations follow directly. *Integrity* is checked by recomputing hashes from the leaves up and comparing the root. *Membership* of a single block is proven with the sibling hashes along its path to the root (a Merkle proof of size $O(\log n)$), without downloading the rest of the module. *Differencing* two versions compares roots top-down and descends only where child hashes differ, so the cost is proportional to what changed, not to the size of the module.

The construction is normative. Roots are only comparable between clients that compute them the same way, so the construction is fixed rather than left to an implementation. A conforming

implementation MUST use the Merkle Tree Hash of RFC 9162, Section 2.1.1 [17], over the module's block identifiers, with two properties that matter for security and for the guarantees claimed above:

- **Domain separation.** Leaves are hashed with a `0x00` prefix and internal nodes with `0x01`, so a leaf hash can never be mistaken for a node hash.
- **Unambiguous splitting.** The tree splits at the largest power of two below n. The naive alternative, duplicating the last node on an odd level, admits a second-preimage attack in which two distinct leaf sets produce the same root [21], which would defeat the whole point of identifying a version by its root.

Leaves are sorted. Leaves MUST be deduplicated and ordered by their digest before the tree is built. Sorting is what makes the root a pure function of the *set* of blocks, which is precisely the claim that two parties who assembled the same blocks obtain the same root. A layout that preserved insertion order would break that claim while looking correct in every single-writer test.

The layout identifier. The construction is named `rfc6962-sorted/1`, and that identifier travels with every composition and module reference. A client that meets a composition built under a layout it does not implement MUST refuse it rather than recompute a root that would not be comparable. The name keeps its historical form: RFC 9162 obsoletes RFC 6962 but defines the same tree, with the same empty hash, the same prefixes, and the same split, so a root computed under either reading is byte-identical. The version suffix is what moves if the construction ever does.

What is persisted. Only the sorted leaf list and the root are stored. Internal nodes are scaffolding derived on demand, so there is nothing to keep in sync, and differencing two versions is a set operation over leaf lists rather than a descent through stored nodes.

Why a DAG rather than a tree. Because blocks are shared across versions. When a new version reuses unchanged blocks, those blocks are shared by hash instead of duplicated, so a node may be referenced by more than one root. The structure is therefore a directed acyclic graph, as in Git's object store. This structural sharing is what makes incremental updates cheap (Section 7): a new version adds only the changed blocks and a new root, and reuses everything else by reference. What sharing guarantees is precisely this reuse of blocks, which are the bytes a consumer would otherwise have to transfer. How much of the DAG's internal structure above the leaves is also reused depends on the layout a conforming implementation chooses, and internal nodes carry no content of their own, being hashes of hashes.

Example

`semantic-root v1` contains `K1`, `K2`, and `K3`. If `K2` is corrected, a new `K2′` is created and `semantic-root v2` references `K1`, `K2′`, and `K3`. `K2` is never modified in place, and `K1` and `K3` are shared between the two roots (Figure 4).

Two Merkle DAGs. It is worth separating the *internal* Merkle DAG described here, which versions the blocks inside a module, from the *external* Merkle DAG that OCI itself forms over manifests, descriptors, and blobs to version whole modules (Section 7, Figure 6). The same idea is applied at two levels: blocks within a module, and modules within a brain.

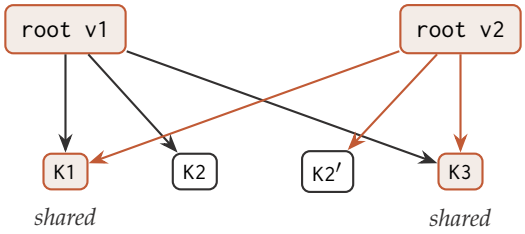

Figure 4: Structural sharing in the internal Merkle DAG. Correcting K2 into K2′ yields a new root v2 that reuses K1 and K3 by hash; only the changed block and the new root are added. Solid dark edges belong to v1, accent edges to v2.

Storage-independent identity. Because the root depends only on the blocks and their structure, it is a canonical identity of a knowledge state, independent of how that state is stored or transported. Two parties that assembled the same blocks obtain the same root and can verify they hold identical knowledge; the OCI digest, by contrast, identifies a particular physical file or manifest.

6.4 The composition document

A Merkle root commits to a set of blocks but *cannot be inverted back into it*. This is easy to overlook and structurally decisive: a version identified by its root alone can be verified and not reopened. Given a root, a client can confirm that a leaf list it already holds is the right one, but it cannot recover that list from the root. Something has to store the leaves.

That something is the *composition document*: the persisted set of block identities that a root commits to. It carries the protocol version, the memory type it belongs to, the Merkle layout that produced the root, and the leaf list in canonical order:

```
{
  "boltzmann":   1,
  "memory_type": "semantic",
  "layout":      "rfc6962-sorted/1",
  "block_ids":   ["sha256:K1", "sha256:K2", "sha256:K3"]
}
```

Storing it is not redundant with the root: the root is the *claim* about a version, and the document is what makes the claim resolvable. This document is what a module layer carries when a brain is published (Section 7), and it is the object every removal mechanism actually operates on. A drop does not mutate a block; it produces a new composition, and therefore a new root (Section 12.3).

A composition MUST be immutable: every operation over it yields a new composition rather than modifying one in place, so that a root already handed to a consumer can never change underneath it. A conforming implementation MUST refuse a composition document whose declared layout it does not implement, and MUST refuse one whose declared protocol version it does not implement.

6.5 The snapshot

A module root identifies one module. A brain is several modules at once, and nothing said so far names the combination. The *snapshot* is that document: it states, for each installed module, which version of it this brain currently is.

```
{
```

```
  "boltzmann":  1,
  "modules": {
    "canonical": { "root":            "sha256:AAA",
                   "composition":     "sha256:AAAdoc",
                   "block_count":     412,
                   "layout":          "rfc6962-sorted/1" },
    "semantic":  { "root":            "sha256:CCC2",
                   "composition":     "sha256:CCC2doc",
                   "block_count":     1907,
                   "layout":          "rfc6962-sorted/1",
                   "embedding_model": "bge-m3@1.5" }
  },
  "created_at": "2026-07-24T10:15:00Z",
  "parents":    ["sha256:PREVIOUS_SNAPSHOT"],
  "trust_root": { ... authorized keys and their scopes ... },
  "labels":     { "release": "v8" }
}
```

Each module reference carries the root (its logical identity), the digest of the composition document (so the version can be reopened, not merely verified), the block count, the layout, and, when a vector index travels with the module, the embedding model and version that produced it (Section 6.6).

Five properties of this document do most of the work in the rest of the paper.

It is the unit of atomic change. Adding one semantic block also advances provenance, and a canonical drop can advance three modules at once (Section 12.3). Those are still *one* version of the brain. A conforming implementation MUST advance every affected module in a single successor snapshot rather than one at a time: minting intermediate snapshots that were never published would leave the parent chain pointing at documents no consumer can resolve.

It is a chain, and occasionally a graph. The `parents` field names the snapshots this one succeeds, so the history of a brain is an auditable hash-linked structure rather than a sequence of unrelated roots. This is what a consumer walks to tell whether a remote brain is ahead of, behind, or diverged from a local one (Section 7.4). A linear history carries one entry, which is the ordinary case; a first snapshot carries none; a reconciliation of two histories carries two or more (Section 13). The list is ordered, and the *first* parent is the history a reconciliation was performed onto: every rule elsewhere in this document that speaks of "the parent" of a snapshot means that one.

It is a subset, not necessarily the whole. A brain may hold only some modules, because selective installation is the point of packaging each module separately (Section 7). A snapshot therefore names the modules that are installed, and an operation against a module that is absent MUST fail with a distinguishable error rather than be treated as an empty module.

Which level of hash identifies a snapshot. The snapshot document is a transportable file, so its own identity is an OCI digest, not a fourth kind of hash (Section 6.8). It is made *of* Merkle roots and is identified *by* an OCI digest, and keeping those two straight is the whole point of the three-level separation.

It carries the trust root. The `trust_root` member holds the keys authorized to sign for this brain and the scopes each one holds (Section 8.4). It lives here rather than in a module or a layer because it MUST reach every install, complete or partial, and because carrying it inside the signed document means a signature can never be evaluated against a key list the signer did not commit to.

Its timestamp decides nothing. The `created_at` member is descriptive. It is written by whoever built the snapshot, so a conforming implementation MUST NOT use it for any security or ordering decision. Ordering comes from the parent chain, which cannot be altered without changing every descendant (Section 8.5).

6.6 Indices

Indices are derived: they can be rebuilt from the blocks. They may travel inside the same module so that each consumer need not recompute them, but they never replace the content. Table 6 lists the indices and their roles.

Table 6: Indices, their typical queries, and their role.

Index	Typical query	Role
Hash map	Get exactly `sha256:ABC`.	Immediate physical resolution.
B^+ tree	Episodes between two dates; concept by ID.	Ordered keys and range scans.
Inverted	Blocks containing “Fourier”.	Lexical matching.
Vector	“Decompose a periodic function into sines”.	Semantic similarity.
Graph	Related concepts, evidence, dependencies.	Associative traversal.
Bitmap / facets	`semantic AND class=signals AND current`.	Efficient filter intersection.
Catalog	Everything filed under “Parciales”.	Navigating evidence by axis.

Whether an index travels inside the module or is rebuilt on installation depends on whether it can be regenerated without a model. Structural indices (hash map, B^+ tree, inverted, bitmap, and, when relations live on the blocks, the graph) are deterministic functions of the blocks and can be rebuilt cheaply by any client. The vector index is the exception: rebuilding it requires an embedding model, which a model-agnostic client does not carry. The vector index therefore travels with the module and records the embedding model and version that produced it, so that consumers can reproduce or compare it. This keeps the blocks as the single source of truth while acknowledging that one derived structure cannot be regenerated in isolation.

The same argument applies to a client that restarts. A structural index can be rebuilt from the installed blocks whenever it is needed, so losing it costs only the time to recompute it; the vector index cannot, and a client that holds it only in memory has lost it for good. An implementation must therefore persist the travelling index locally, alongside the module it belongs to, and must not republish a placeholder in its stead: an artifact whose layer claims a vector index and carries none is worse than one that omits the layer, because a consumer can detect the absence and cannot detect the emptiness.

A travelling index MUST be bound to the composition it was built over. Three requirements stated elsewhere in this document compose badly if this is left implicit. A travelling index cannot be rebuilt by a model-agnostic client, which is the only reason it travels. A drop rebuilds

the module's indices from the new composition (Section 12.3). And no operation in this protocol removes a single entry from an index, because indices are built and queried, never edited. Put together, an implementation that does the obvious thing publishes, after a drop, an index that still contains the representation of the dropped block.

Queries stay correct, because a hit that is no longer a member fails membership verification and is filtered out (Section 11). That is what makes this worse rather than better: it is not a retrieval bug, so nothing surfaces to be noticed, and the published artifact silently carries a learned representation of content that was deliberately excluded. For a drop motivated by law or safety that leaks precisely what the drop was meant to remove, and text embeddings recover enough of their input for the concern to be a practical one [22]. This is the *ordinary* path, not the exceptional one: Section 12 presents drop as the normal removal mechanism and redaction as the rare exception, and it is the common case that leaks.

Three rules close it. A travelling index MUST record the module root it was built over, so that staleness is detectable rather than invisible. An implementation MUST NOT publish an index built over a composition other than the one its module names: a model-bearing client rebuilds the index, and a client that cannot rebuild it MUST omit the layer rather than ship a stale one, which is the argument of the previous paragraph carried one step further. And a consumer that meets an index bound to a root other than the installed one MUST report it as stale and MUST NOT answer queries from it. Removing individual entries is permitted where an engine supports it and stays an implementation matter rather than a protocol operation, because deletion from an approximate nearest-neighbour structure is engine-specific and lossy. What the protocol requires is narrower and checkable: a published artifact MUST NOT claim an index it did not build over the composition it ships.

This separation has a representational reading. A block carries meaning in two portable, *symbolic* forms: its text and its explicit relations to other blocks, the edges that in aggregate form the knowledge graph. Learned, *sub-symbolic* representations (embeddings) are never stored in the block; they live in the vector index as derived, model-tagged views. Operating mathematically on concepts, such as vector arithmetic or steering, happens inside the external LLM in its own representation space, not in the brain. The brain thus owns durable symbolic knowledge and borrows the operational vector space from whichever model queries it. This also clarifies a naming coincidence: the *semantic* module denotes general, consolidated knowledge (as opposed to episodic memory), not an embedding-based representation.

6.7 The catalog: making canonical evidence navigable

A canonical module that holds five hundred sources is, to a consumer, five hundred hashes. The protocol says what each one is and proves that it is intact, and it says nothing at all about where any of it belongs. A brain published that way is a pile, and the party best placed to organize it, the one who assembled it, has no way to record the organization so that it travels with the artifact.

Section 11 already assumes otherwise. Its request carries a `subject` filter, which until now was free text matched against nothing this document defines. This subsection supplies what that filter needs.

Where classification cannot live. Two placements are ruled out before any design begins, and both follow from constraints stated earlier.

It cannot live inside the canonical block. A canonical block carries only what its bytes determine, so that re-registering an identical original is a no-op (Section 5). A classification written into the payload would enter `block_id`, two parties filing the same PDF under different headings would compute different identities, and the evidence would fork every time someone changed their mind about where it goes.

It cannot live in provenance either, and the reason is practical. A processing task MUST NOT allow candidates in the canonical or provenance modules (Section 10), so a classification held there could never be proposed by a model. Filing a corpus of any size by hand is not a workable requirement.

Classification is consolidated knowledge. What remains is the correct answer and not merely the surviving one. "This source is about Fourier analysis" is a claim derived from reading the source, which is precisely what the semantic module holds and precisely what the ingestion path produces. Placing it there means a model may propose it, the validation of Section 10.3 judges it, a drop removes it, and `rederive` rebuilds it. No new machinery is introduced.

Three kinds of block. A *scheme* declares an axis of classification and whether that axis is exclusive. A *class* is a node on one axis, carrying a label and the scheme it belongs to. A *placement* files one canonical block under one class, and cites that canonical block as its evidence, which is what makes the canonical cascade of Section 12.3 apply to it without a special rule: dropping a source removes its filings in the same commit, and nothing has to remember to clean up.

```
scheme     { "kind": "scheme", "scheme": "subject", "exclusive": false }
class      { "kind": "class",  "scheme": "subject", "label": "Fourier analysis" }
placement  { "kind": "relation", "evidence": ["sha256:PARCIAL_2024"],
             "relations": [{ "predicate": "classified_as",
                             "target": "sha256:CLASS_FOURIER" }] }
```

Hierarchy is a block, not a field. A class does not name its parent. The narrowing relation between two classes is its own block, naming both endpoints, and the asymmetry with a placement is deliberate: a placement takes its source from `evidence` and needs one edge, while a narrowing has two class endpoints and no evidence at all.

Recording the parent as a field on the class would put it inside that class's `block_id`. Moving a node in the taxonomy would then change the node's identity, every placement pointing at the old identity would dangle, and reorganizing one heading would invalidate everything filed beneath it. Reorganization is routine, so it must be cheap. This is the same reason supersession is a provenance edge and not a field on either block (Section 10).

One axis is never enough. A source on the Fourier transform is mathematics and is signal processing, and a past exam is also a past exam and also from 2024. Forcing a single tree means choosing which of those questions a consumer is allowed to ask. Library classification reached this conclusion long before content addressing existed, and answered it with facets: a document takes coordinates on several independent axes instead of one shelf position [31]. A brain therefore MAY declare several schemes, and a canonical block MAY hold placements on each of them. The `broader` and `narrower` vocabulary here is the one SKOS uses for the same purpose [20].

The catalog index. The classification blocks are the source of truth, in the sense Principle 1 requires, and the *catalog* is the derived index over them: what is filed under a class, which classes a source holds, and what the axes look like.

Unlike the vector index of Section 6.6, a catalog is a deterministic function of blocks a client already has, so any client rebuilds it and no model is required. It MAY nonetheless travel with the canonical module, bound to the composition it was built over under the staleness rule of Section 6.6, so that a consumer installing only canonical receives a navigable brain instead of a pile.

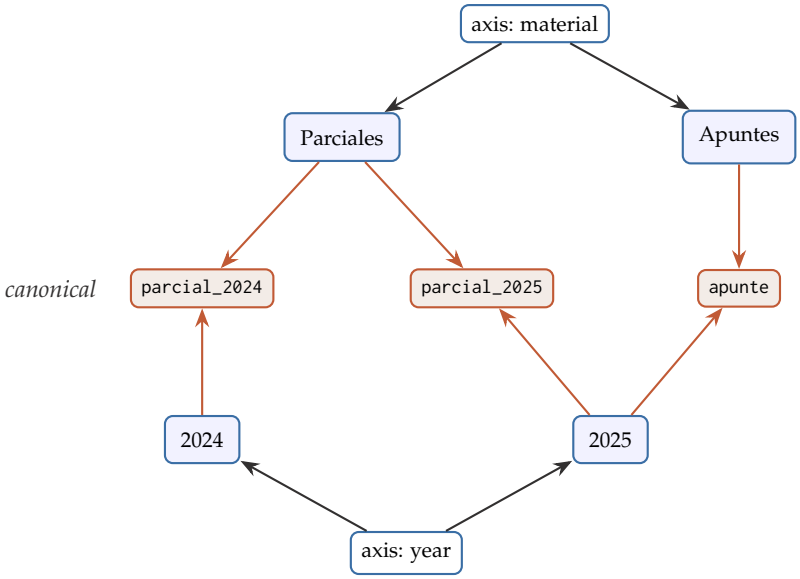

Figure 5: Two axes over the same three sources, grouping them differently: by material the sources split {parcial_2024, parcial_2025} against {apunte}, and by year they split {parcial_2024} against {parcial_2025, apunte}. Every accent edge is one placement block. No canonical block is copied and none of them changes, so adding, removing, or restructuring an axis touches only the semantic module and the canonical root a consumer already holds stays exactly as it was.

Normative rules. A canonical block MUST NOT carry classification. Reclassification MUST NOT change any canonical composition, so a consumer that already holds the evidence transfers nothing when a publisher reorganizes it. A class MUST NOT name its parent as a payload field. A placement MUST cite the canonical block it files as evidence. Where a scheme declares itself exclusive, a second placement for the same block on that axis MUST yield `CONTRADICTED` rather than silently coexist. And this document defines no scheme of its own: which axes exist, and what is on them, is a decision each brain records, in the same way it records a retention policy.

What this buys, and what it costs

The evidence is untouched, so reorganizing the whole library moves no bytes and invalidates no root. One source appears on as many axes as it deserves with no copy of it anywhere. A drop removes its filings through the cascade that already exists. Against that: classifying a large corpus for the first time is one model pass per source, and a consumer that installs no semantic module and receives no travelling catalog still sees the pile.

6.8 Three levels of hashes

A recurring source of confusion is that the same cryptographic algorithm is used for three different kinds of identity. Table 7 separates them.

Table 7: Three levels of hashes.

Level	What it identifies	Example
Block hash	A logical unit of knowledge.	`sha256:FOURIER_CONCEPT`
Merkle root	The logical composition of a module.	`sha256:SEMANTIC_ROOT_V7`
OCI digest	Any transportable file: a published blob, a manifest, or one of the protocol's own documents.	`sha256:OCI_MODULE_BLOB`

These values may use the same algorithm, but they do not represent the same thing. The block hash is *knowledge identity*; the Merkle root is the identity of a *logical snapshot*; the OCI digest is the identity of a transportable *file or manifest*.

The composition and snapshot documents of Sections 6.4 and 6.5 are worth locating explicitly on this scale, because both are made of lower-level identities while being identified at the third level. A composition document *contains* block hashes and *is* an OCI digest; a snapshot document *contains* Merkle roots and composition digests and *is* an OCI digest. Neither introduces a fourth level. The rule is that a level is determined by what a value identifies, not by what it is computed over.

6.9 Schema versions and forward compatibility

Block schemas evolve: a memory type acquires a field, and blocks written afterwards are not shaped like the ones written before. Because `schema_version` sits inside the envelope, and therefore inside `block_id`, this is not a cosmetic concern. Three rules keep an evolving brain readable.

A version is a statement, not a preference. A block MUST be written under the *oldest* registered schema its payload satisfies, and the proposer of a block does not get to choose. Writing under the newest schema instead would mean that introducing a schema silently re-versions every block written afterwards, including blocks that use nothing the new schema added, making an artifact unreadable to consumers that had not upgraded in order to record knowledge those consumers could have read perfectly well. Oldest-that-fits inverts this: a brain stops being readable by an older client only at the point where it genuinely uses something that client has no schema for.

An unknown version MUST fail loudly. A block declaring a schema version a client does not implement was written by a newer implementation. The schema cannot be inferred from the bytes, so the client MUST refuse the block and say so, rather than parse what it recognizes and drop the rest.

The refusal must be possible before the download. Which schema versions a module holds is declared on the published artifact (Section 7). The information was always *in* the brain, since every envelope names its version, but only reachable by fetching and parsing blocks, which is to say after a download the consumer may not be able to use. Declaring it up front lets a client refuse a brain it cannot read before transferring a byte, and say why.

This is a different question from the protocol version, and the two MUST stay separate. The protocol version asks whether this is a Boltzmann artifact at all; the schema version asks whether this client has the schemas for the knowledge inside it. A brain using a schema a consumer lacks is still a brain, so that consumer MUST still be able to install the modules it *can* read, which a protocol-level refusal could not express.

7 Distribution with OCI Artifacts

OCI is a standard for content-addressed manifests, descriptors, and blobs. A compatible registry stores the blobs and allows referencing them by tag or digest. The specification organizes its components as an external Merkle DAG of descriptors [25, 26, 29]. Figure 6 shows how OCI distributes modules while the internal Merkle versions the blocks within each module.

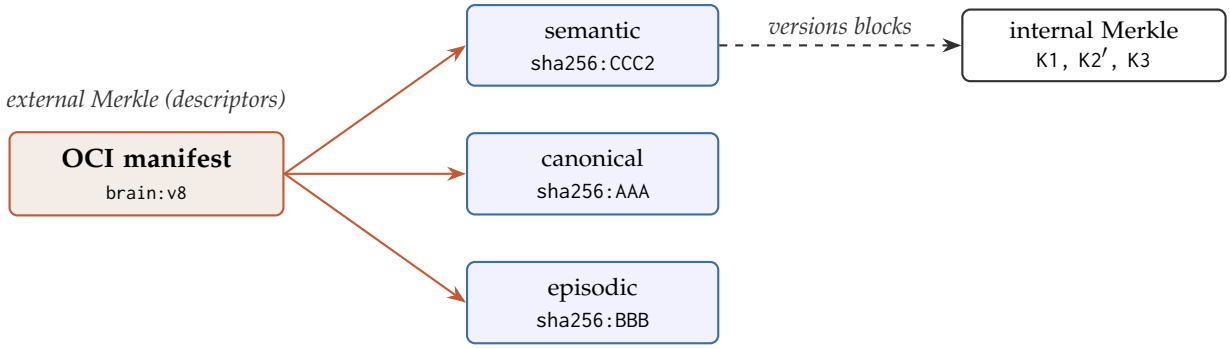

Figure 6: OCI distributes modules as separate blobs referenced by a small manifest; inside each module, an internal Merkle DAG versions the individual knowledge blocks.

7.1 Complete brain

A brain is published as an OCI Artifact whose *config blob is the snapshot document itself* (Section 6.5) and which carries one layer per module:

```
manifest  artifactType: application/vnd.gaussia.boltzmann.brain.v1+json
  config  -> the snapshot document
  layers
    canonical  -> sha256:AAA   annotations: memory-type, merkle-root,
    episodic   -> sha256:BBB                merkle-layout, block-count
    semantic   -> sha256:CCC
    procedural -> sha256:DDD
    provenance -> sha256:EEE
    (vector index layers, when one travels)
```

The OCI manifest is small, and downloading it does not imply downloading any module. Making the snapshot the config blob means the same document is both the local state and the wire format, so publishing is a transfer rather than a conversion. It also means the trust root of Section 8.4 arrives with every install, since there is no install that does not fetch the config. Signatures are deliberately absent from this listing: they are published as separate artifacts referring to the manifest, for reasons Section 8.8 gives.

Media types. Two clients that disagree on media types cannot pull each other's brains, so these are normative here (Table 8).

Table 8: Normative media types.

Role	Media type
Artifact type	`application/vnd.gaussia.boltzmann.brain.v1+json`
Config (snapshot)	`application/vnd.gaussia.boltzmann.snapshot.v1+json`
Module layer	`application/vnd.gaussia.boltzmann.module.{type}.v1.tar+gzip`
Vector index layer	`application/vnd.gaussia.boltzmann.index.vector.v1+bin`
Signature record	`application/vnd.gaussia.boltzmann.signature.v1+json`

A module layer is a tar archive holding the module's composition document and its block bytes. Both the tar and the compression stream MUST be written deterministically, so that two clients packing the same module produce byte-identical layers. Without that, registry deduplication silently stops working and every publish transfers blobs the registry already

holds. Gzip is specified rather than a denser codec because it is universally available: a client that needed an extra dependency to read a published brain would be trading portability for a few percent.

Annotations. The manifest carries what a consumer needs to decide *before* downloading (Table 9).

Table 9: Manifest and layer annotations, all prefixed `ai.gaussia.boltzmann`.

Annotation	What it declares
`memory-type`	Which module a layer holds. This is what a selective install resolves on.
`merkle-root`	The layer's internal Merkle root.
`merkle-layout`	The construction that produced the root; roots are comparable only between clients that agree here.
`block-count`	How many blocks the composition holds.
`protocol-version`	Which protocol version the artifact conforms to.
`schema-versions`	Which block schema versions each module holds (Section 6.9).
`embedding-model`	Model and version behind a vector index layer.
`index-kind`	Which kind of index a travelling index layer carries.
`source-snapshot`	The publisher's full snapshot this artifact was projected from.
`trust-root`	Digest of the trust root carried in the snapshot, so a consumer can compare it against a pin before transferring anything (Section 8.7).
`index-root`	The module root a travelling index layer was built over (Section 6.6).

The `merkle-root` annotation deserves emphasis because it is deliberately distinct from the descriptor's own digest, and that distinction is the third level of Section 6.8 made operational. The digest identifies the transportable file; the root identifies the logical composition inside it. Two registries holding the same brain agree on digests while knowing nothing about modules or snapshots, which is exactly the gap Section 4.3 identified in composing OCI with a retrieval stack. This annotation closes it.

The `source-snapshot` annotation handles publishing a subset of modules. Such an artifact's config carries a *reduced* snapshot, which by construction is not in the publisher's own history, so without recording the snapshot it was projected from, pushing a projection back to the same tag would look like a divergence when nothing diverged.

7.2 Selective installation

To install only the episodic module, the client resolves the descriptor marked as `episodic` and downloads `sha256:BBB`. It does not need to download `canonical`, `semantic`, or `procedural`. It then opens the episodic Merkle DAG and its local indices.

Necessary condition

Selection is only efficient if each module is a separate OCI blob or artifact. If everything is mixed into a single file, the whole file must be downloaded.

7.3 Incremental update

If only the semantic module changes, the new manifest can reuse the digests of the remaining modules. The consumer downloads only the new semantic blob; inside it, the Merkle root reveals which logical blocks changed.

Table 10: An incremental update touches only the changed module's digest.

Version	Canonical	Episodic	Semantic	Procedural
v7	AAA	BBB	CCC	DDD
v8	AAA	BBB	CCC2	DDD

OCI contributes storage, registry authentication, transport, digest-based deduplication, and tags. Boltzmann Brain contributes the meaning of the modules, the blocks inside them, and their query contract.

7.4 Local state, atomicity, and divergence

Everything described so far is content-addressed and immutable. A working brain needs exactly one piece of mutable state: a pointer saying which snapshot is current. Isolating it to a single pointer is what makes a commit atomic, and it is the only place where the ordering of writes matters.

Commit is a pointer move. A conforming implementation MUST write every blob a new snapshot references before moving the pointer, and MUST move the pointer last. Blobs written and never referenced are unreachable and will be reclaimed by pruning (Section 12); a pointer moved before its blobs exist would name a state that cannot be resolved. The asymmetry is the whole reason a commit can be interrupted without corrupting a brain.

Retained roots. Alongside the current snapshot, a brain keeps a set of retained snapshots: tagged releases plus a bounded number of recent ones. This set is what reachability is computed from when pruning reclaims blocks (Section 12), and it is what gives "a retained root" a precise meaning.

Origin. A brain pulled from a registry MUST record where it came from: the repository reference, the tag, the snapshot the remote was on at pull time, and whether the install was partial. The recorded remote snapshot is what a later publish compares against. Content addressing alone would not protect a publisher here: overwriting a tag leaves every blob in place, but if no retained root names them, the work is gone in the only sense that matters.

Divergence is detected, not resolved. Before publishing, an implementation MUST compare the remote snapshot against the local chain. If the remote is an ancestor of the local snapshot, the publish fast-forwards. If it is not, the two brains have diverged, and the implementation MUST refuse the publish and report where the histories parted, rather than overwrite.

Refusing is where this subsection's obligation ends, not where the protocol stops. Detecting divergence is a reader's duty and belongs here; deciding what to do next is a separate question, and Section 13 answers it. Because a snapshot names its parents as a list, a reconciliation is representable, and because a module composition is a set of immutable blocks, most of one is mechanical. Two cases this subsection would otherwise leave stranded are resolved there rather than by hand: a diverged history becomes a snapshot with two parents, and a partial install is published back as a reconciliation in which the modules it never fetched take their roots from the remote unchanged (Section 13.7).

8 Authenticity and Trust

Everything specified so far establishes that a brain is *intact*. Nothing specified so far establishes *who assembled it*. Those are two different claims, and this section separates them, identifies the single assertion the hash structure cannot make on its own, and fixes the smallest structure that can carry it.

8.1 What the hash structure already proves, and the one thing it cannot

The protocol stacks four commitments, each one pinning the level below it. A `block_id` commits to a block's bytes, including the memory type, schema version, and serialization named in its envelope (Section 6.1). A Merkle root commits to the exact set of block identifiers in a module (Section 6.3). A composition document makes that set *resolvable* rather than merely verifiable, and is itself content-addressed (Section 6.4). A snapshot commits to one root per module, and to its parents by digest (Section 6.5).

Because the parents are named by digest, the chain of snapshots is itself hash-linked. Altering anything anywhere, a byte in a source, a block's membership in a module, an entry in an old snapshot, changes an identifier, which changes every value above and after it.

Two consequences follow, and they point in opposite directions.

Integrity is already total and transitive. Any consumer, offline, with no configuration and no trust in anyone, can recompute the whole structure and confirm that a brain is exactly the brain those identifiers describe. Nothing needs to be added for that, and nothing in this section may be allowed to make it conditional.

Authorship is unreachable from inside. Every check above is a check of internal consistency, and internal consistency is cheap to manufacture. Anyone may assemble a brain containing whatever they like, compute its roots correctly, and publish a snapshot that passes every verification this document defines. The attack requires no cleverness: copy a brain, add a fabricated semantic block, recompute the affected roots, publish a consistent snapshot under someone else's name. The structure proves that a brain is coherent. It cannot distinguish a coherent brain from *a particular party's* coherent brain.

This is worse than a missing feature, because without it a guarantee stated elsewhere in this document does not hold. Principle 8 requires that every drop, cascade, and redaction be auditable, and Section 12.5 rests on provenance being the ledger that makes exclusion trustworthy. But provenance is made of blocks like everything else, and the actor it records is a *declared identifier* rather than an authenticated identity. Whoever can write to a brain can forge its audit trail. A signature is what makes that actor field mean anything, which is why authenticity is specified here rather than left to a deployment.

So what is missing is not more hashing. It is a single assertion, by a party, that the hashes cannot make on their own.

8.2 Where a signature attaches

The hash structure determines this almost completely, and the answer is unusually economical: **one signature over the snapshot covers everything**.

Signing a snapshot commits to the module roots it names. Each root commits to that module's entire block set. Each block commits to its own bytes. The parent digests commit to the whole prior history, since changing any ancestor would change its digest and break the chain leading to the signed value. A conforming implementation MUST therefore attach

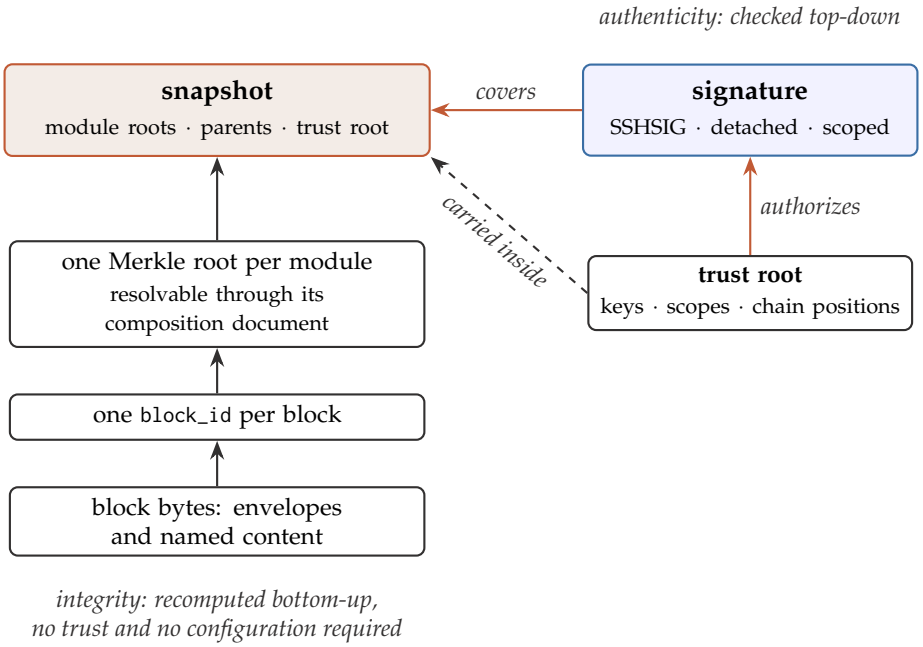

Figure 7: The two verifications and where they meet. The left column is recomputed from the bytes upward and needs no trust in anyone; the snapshot is the only place a signature attaches, because it is the only document that commits to every module root at once. The trust root travels inside the snapshot and therefore inside the signed bytes.

signatures to snapshots, and MUST NOT require a signature over any other object to establish authorship. Signing the head is sufficient; signing every historical snapshot is OPTIONAL.

A useful side effect: because the snapshot names each module's Merkle layout and each block envelope names its serialization, a signature transitively commits to *how* the values were computed and not merely to the values. A consumer cannot be steered into recomputing under a different construction and finding a mismatch it has no way to interpret.

Blocks MUST NOT carry the signer. This follows from an argument this document already makes. Block identity must depend on the hashed bytes alone, or re-registering an identical source stops deduplicating (Section 5). If a signer entered the hashed content, two parties ingesting the same PDF would produce different block identifiers and structural sharing would collapse. Blocks stay anonymous and immutable; snapshots carry attribution. Who acted is recorded in provenance and authenticated by the signature over the snapshot that committed the act.

Signing module roots individually is not a substitute. Five independent signatures are five independent claims, and nothing binds them into a single state. An attacker who holds five separately signed roots can combine today's semantic root with yesterday's canonical root, the one that still contained a source since dropped, and both signatures verify. The consumer installs a brain that never existed, holding evidence that was deliberately excluded, with every signature checking out. Only a signature over the document that *combines* the roots prevents this, which is precisely what a snapshot is. A conforming implementation MUST NOT accept per-root signatures in place of a signature over a snapshot.

8.3 The signing mechanism

Signatures are made with the SSH signature format, SSHSIG [28], the same mechanism Git uses when configured with `gpg.format=ssh`. There is no PKI and no certificate authority: contributors already hold SSH keys, and the forges already publish them. Ed25519 [13] is the RECOMMENDED key type.

A signature MUST be produced over the canonical bytes of the snapshot document (Section 6.2) under the namespace `boltzmann.snapshot.v1`. The namespace is not decoration: it is what prevents cross-protocol replay, so that a signature a contributor produced for a Git commit cannot be presented as a Boltzmann signature, and a later version of this document can introduce a new namespace without invalidating signatures made under this one. A conforming implementation MUST reject a signature made under any other namespace.

Non-goal: key management

The private key never enters the protocol. It stays where SSH keys already live: an agent, a hardware token, or a key-management service. The brain does not manage keys, for the same reason it does not embed a language model. A conforming implementation MUST NOT require a private key to be stored inside a brain, and MUST NOT define a format for one.

Signatures are detached. This is deliberately unlike Git, which embeds the signature inside the commit object and therefore changes the commit's identity when it is re-signed. Keeping signatures outside the document they cover buys three things that this architecture specifically needs:

- Snapshot identity does not depend on whether, or by whom, a snapshot was signed. Two parties who assembled the same knowledge still compute the same snapshot digest.
- A snapshot committed locally without a signature becomes signed at publication *without changing identity*, which matters because Section 7.4 already separates local state from publication.
- Several signatures can cover one snapshot. This is the only way to express a quorum, and it is the reason the choice is not merely stylistic.

A signature record names what it covers, under which namespace, by which key, and which scopes it claims:

```
{
  "boltzmann": 1,
  "snapshot":  "sha256:SNAPSHOT_DIGEST",
  "namespace": "boltzmann.snapshot.v1",
  "key":       "SHA256:5tS7wqf2K...",
  "scopes":    ["commit"],
  "signature": "-----BEGIN SSH SIGNATURE----- ..."
}
```

The `key` field is the signing key's SSH fingerprint, and it is an index rather than an authority: SSHSIG carries the public key inside the signature blob, so a verifier MUST check that the key inside the blob matches an authorized entry of the trust root, and MUST NOT trust the named fingerprint on its own. A record whose fingerprint and embedded key disagree MUST be rejected.

8.4 The trust root

A signature answers *who*. It does not answer *and why should that matter*. Answering the second requires knowing which keys are authorized for this brain, and a consumer pulling from a mirror, a cache, or a tarball MUST be able to learn that without reaching a server, or the offline property of Section 8.1 is lost the moment authenticity is introduced.

The authorized keys therefore travel *inside* the brain, as a *trust root* carried in the snapshot document under the key `trust_root`:

```
{
  "boltzmann":     1,
  "revision":      3,
  "namespace":     "boltzmann.snapshot.v1",
  "govern_quorum": 2,
  "keys": [
    { "key":              "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...",
      "scopes":           ["ingest", "commit", "drop:canonical", "govern"],
      "since":            1 },
    { "key":              "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...",
      "scopes":           ["ingest", "commit", "govern"],
      "since":            1 },
    { "key":              "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...",
      "scopes":           ["propose"],
      "since":            2 },
    { "key":              "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...",
      "scopes":           ["commit"],
      "since":            1,
      "retired_from":     3 },
    { "key":              "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5...",
      "scopes":           ["commit"],
      "since":            1,
      "compromised_from": "sha256:SNAPSHOT_S41" }
  ]
}
```

Its identity is the digest of its canonical serialization, and that digest, rather than any individual key, is what a consumer pins (Section 8.7).

Two forms of position, and why they are not interchangeable. `since` and `retired_from` name a *revision number*; `compromised_from` names a *snapshot digest*. The asymmetry is a construction constraint rather than a preference. A revision travels inside the snapshot that introduces it, so naming that snapshot by digest from within the trust root would be circular: the digest cannot be computed until the trust root is complete, and the trust root would not be complete until it named the digest. A revision number breaks the cycle and gives up nothing, because a revision corresponds to exactly one snapshot in the chain and the position is recoverable by walking parents. A `compromised_from`, by contrast, always names a position that is already published, so its digest is known and no cycle arises. The finer granularity is necessary rather than incidental, since a revision may be in force across many snapshots and a key compromised in the middle of that span cannot be described by a revision boundary.

One consequence is worth making normative, because it removes an obvious lie. A revision that admitted a key is itself in the chain, so `since` is *confirmable* rather than merely asserted: a verifier walking revisions sees when a key first appeared. A trust root claiming a `since` earlier

than the chain supports MUST be rejected, which is what stops a key that has just added itself from also claiming to have been authorized from the beginning.

Why it is not a sixth module. Modules are selectively installable (Section 7.2), so a consumer who installed only the semantic module would not receive the very document needed to verify anything at all. The key list has to be present in every install, complete or partial, which places it at the brain level rather than among the modules. Carrying it in the snapshot, which is the brain-level document and the config blob of a published artifact, makes omitting it structurally impossible: there is no install that does not fetch the snapshot. As a second consequence the list falls inside the signed bytes, so a signature can never be evaluated against a key list the signer did not commit to.

Scopes are derived from operations, not invented. A scope names something the protocol already defines rather than a generic role, so that a consumer can check a claim mechanically (Table 11). This is what operationalizes Principle 2: "dropping canonical evidence is allowed only under explicit policy" today names a policy that a consumer has no way to confirm was honoured, and `drop:canonical` is what turns it into a checkable statement about who was permitted to do it.

Table 11: Signing scopes, and how a verifier decides which one a snapshot required. The requirement is computed from the difference between a snapshot and its first parent, never taken from what the signature claims.

Scope	Authorizes	Required when, relative to the first parent
`ingest`	Preserving a source as canonical evidence.	The canonical composition gained blocks.
`commit`	Adding derived blocks; ordinary drops, supersession, and demotion.	A non-canonical composition changed.
`drop:canonical`	Excluding canonical evidence, with its cascade.	The canonical composition lost blocks.
`redact`	Destroying bytes a retained root still names.	Blocks became tombstoned.
`govern`	Revising the trust root.	The trust root digest changed.
`propose`	Offering a snapshot for someone else's consideration.	Never sufficient for a published head (Section 13.6).

An ordinary drop needs only `commit` while a canonical drop needs its own scope, and the asymmetry is the one Principle 2 already draws: an ordinary drop is recoverable by a later commit that re-adds the block, whereas excluding canonical evidence forfeits re-derivation from that source.

A verifier MUST compute the required scope set from the difference between the snapshot and its first parent, or, for a snapshot that has no parents, from its difference against the empty brain (Section 8.6), and MUST reject a signature whose key does not hold every scope in that set, even if the signature claims fewer. The `scopes` field of a signature record is therefore a statement of intent that aids diagnosis, never the basis of the decision. This mirrors the rule of Section 6.9: a version is a statement about what the payload uses, not a preference, and a scope is a statement about what the snapshot did, not a claim the signer gets to make.

The trust root is not access control

Nothing can stop someone modifying a copy of a brain they already hold. The brain is data on their disk, and a field claiming otherwise would be honoured only by clients that chose to honour it. The trust root is a rule the *reader* applies. It does not prevent writing; it prevents *impersonation*. Preventing writes to a particular repository is the registry's job, and the two mechanisms defend against different attackers: registry permissions protect a repository, signatures protect an identity everywhere, including after the bytes have left that repository.

Changing the trust root requires a quorum. If the key list were just another document, whoever can sign could add themselves to it and the structure would authorize itself. A *trust-root revision* is therefore a snapshot that has a first parent and whose `trust_root` digest differs from that parent's, and it MUST satisfy two conditions: it MUST be covered by at least `govern_quorum` valid signatures from distinct keys, and each of those keys MUST hold the `govern` scope in the trust root as it stood *before* the change. Both halves matter, and evaluating against the previous revision is the half that is easy to lose. This is the root-rotation rule of The Update Framework [36], which is the studied answer to exactly this problem.

A trust-root revision MUST NOT change anything else: its module roots MUST equal its first parent's. Governance is thereby a separate, individually auditable event in the chain rather than something folded into a commit, and a consumer can enumerate every change of authority a brain has ever undergone by walking the parents and comparing one digest.

A `govern_quorum` of 1 already stops a stranger, because a stranger holds no listed key at all. A quorum of 2 or more is what protects a brain against a single stolen legitimate key, which is the case worth designing for.

8.5 Retirement and revocation, without clocks

Removing a key and repudiating a key look similar and must behave oppositely.

When someone leaves a project, everything they signed while authorized SHOULD remain valid. If removal were retroactive, a routine departure would invalidate every snapshot that person ever signed, and large stretches of history would stop verifying for an administrative reason.

When a key is compromised, the opposite is required: everything it signed from the point of compromise onward MUST stop being trusted, even though it was signed while the key was still listed.

Separating the two requires knowing *when* a snapshot was signed, and the obvious approach fails. A timestamp inside a signature is written by the signer, so an attacker holding a stolen key can backdate freely; Git has the same hole. The `created_at` field of a snapshot is no better, being written by whoever built it.

The hash structure supplies the missing ordering. Snapshots are chained by digest, so a snapshot's position in the chain cannot be forged: inserting or moving one would change every descendant's parent digest, and those are already published. Validity is therefore expressed *positionally* rather than temporally, and the question a verifier asks is:

> Was this key authorized, with the scope this change required, in the trust root in force at this position in the chain?

Because trust-root revisions are themselves snapshots in that chain, a verifier walks back from a snapshot to find the revision in force and answers the question with no clock involved.

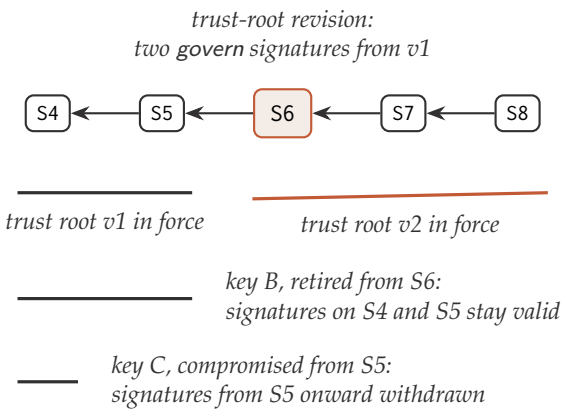

Figure 8: Validity is positional. A trust-root revision is an ordinary snapshot in the chain that changes the key list and nothing else, so the revision in force at any position is found by walking parents. Retirement closes an interval without disturbing what came before it; a compromise record closes one at a position that may precede the revision recording it, and is the only construct that withdraws a signature already accepted.

Two record forms express the two intents, and a conforming implementation MUST distinguish them:

`retired_from` The revision from which the key is no longer authorized. Signatures at earlier positions remain valid. Retirement can never invalidate a signature that was valid before it, which is what makes an ordinary departure harmless.

`compromised_from` A *snapshot* from which the key's signatures are no longer trusted. It MAY precede the revision that records it, and it need not fall on a revision boundary, because compromise is discovered after the fact and in the middle of a revision's span. This is the only construct in this document that invalidates a previously valid signature, and a verifier MUST report it as such rather than as an ordinary authorization failure.

Stating that distinction plainly matters in operation: "signed by a retired key, still valid" and "signed by a compromised key, no longer trusted" are different facts about the same snapshot, and collapsing them either breaks history or hides an attack.

This is the clearest case of the two layers composing. Hash-linking was introduced in Section 6.5 for versioning, and it turns out to supply exactly the unforgeable ordering that revocation needs. Certificate Transparency reaches for the same property through the same structure [17].

8.6 Creating a brain: the genesis snapshot

Every rule stated so far is expressed against a snapshot's first parent, and one snapshot in every brain has none. The `init` operation (Table 14) produces it, and because it is the origin of every authority the brain will ever have, its rules are stated here.

A *genesis snapshot* is a snapshot whose `parents` list is empty. A brain MUST have exactly one, and every other snapshot MUST be reachable from it by following parents. Its module compositions are usually empty, and an empty composition has a well-defined root (Section 6.3), so a brain with no knowledge in it is still a brain that verifies.

Three rules follow, and each closes a gap the parent-relative phrasing would otherwise leave open.

Its difference is taken against the empty brain. A verifier computes the scopes a snapshot required from its difference with its first parent (Section 8.4). For a genesis snapshot that difference is taken against a brain with no modules and no trust root, so every scope its content implies is required at once: `govern` because it establishes a trust root, and `ingest` or `commit` if it also carries blocks. A single self-signing key must therefore hold all of them, which is worth knowing before creating a brain with a narrowly scoped key.

The quorum rule does not govern it, and this is the only exception. A trust-root revision must be signed by keys authorized in the previous revision (Section 8.4), and a genesis snapshot has no previous revision to draw on. It is therefore not a revision, and a verifier MUST NOT reject it for failing the quorum rule. This is the single point in the protocol where authority is asserted rather than derived, and Section 8.7 is entirely about what to do with that fact.

It SHOULD nonetheless satisfy its own declared quorum. A genesis snapshot SHOULD carry at least `govern_quorum` valid signatures from distinct keys listed in its own trust root. This proves nothing against an attacker, who can satisfy it just as easily, and it is not required for that reason. It is required for coherence: a brain that declares a quorum of two and whose founding act carried one signature is stating a rule it has already departed from, and an operator reading that history later cannot tell whether the second holder was ever real. An implementation MAY warn when a genesis snapshot carries fewer signatures than the quorum it declares.

A genesis snapshot MAY be unsigned, and a brain whose genesis is unsigned is a brain with no authorship from the outset. That is the zero-configuration case of Section 8.1, and it remains fully verifiable for integrity. What it cannot later acquire is a signed history: signatures attach to snapshots, and the snapshots that were committed unsigned stay unsigned.

8.7 Bootstrap: the one thing that comes from outside

The quorum rule cannot apply to the first trust root, because there is no previous revision to draw a quorum from. The first list is self-signed, and read from inside the artifact it proves nothing: it asserts that a key is authorized, and the entity asserting it is that key. An attacker can construct an identical-looking object.

This is not a defect specific to this design. Trust cannot be manufactured from inside a system; at some point something must arrive from elsewhere. What a design can do is reduce it to a single decision, taken once. Two mechanisms are specified, and an implementation MUST support both:

Trust on first use, then pin. On first pull the consumer records the digest of the trust root and MUST warn loudly, and by default refuse, on any later change that does not follow the quorum rule of Section 8.4. This is the `known_hosts` model of SSH [43]. It does not prove the first contact was legitimate, but it converts an ongoing exposure into a single moment of exposure.

Pin out of band. The digest is published through an independent channel, a project site, a repository README, or DNS, and compared once by hand. Compromising the distribution then requires compromising two independent things.

Trust on first use is the RECOMMENDED default, with out-of-band pinning available to anyone who wants a stronger anchor. The residual limitation is stated rather than glossed: a consumer who has never seen a given brain has no basis on which to reject an impostor's, and no arrangement of data inside the artifact can change that. Every comparable system carries this problem; SSH lives with it, the web spends an industry on it, and transparency logs are another answer to it [17, 24].

Pinning is also what closes the last impersonation route, which is the subject of Section 8.11.

8.8 How a signed brain is published

The trust root has a home already: it is inside the snapshot, and the snapshot is the config blob (Section 7). Signatures need a different answer, because they must be able to accumulate.

A signature cannot simply be a layer. Adding a second signature to satisfy a quorum would change the manifest, and therefore the brain's digest, so a brain would change identity because someone countersigned it, with no knowledge having changed. That is precisely what detaching signatures was meant to avoid.

The OCI Referrers API exists for this [27]. A signature is published as a separate artifact whose `subject` is the brain's manifest, which is how signature attestations already work [24, 41], so signatures accumulate *around* an artifact without touching it. A conforming implementation MUST publish signatures this way, so that registry tooling can discover them without implementing this protocol.

But referrers exist only inside an OCI registry, and a signature that lives only there does not survive a brain being exported to a tarball, a repository, or a disk, which the portability argument of Section 4.3 will not accept. So both are required. Signature records are part of the brain format and MUST travel with an exported brain, alongside the snapshot; publication additionally mirrors them into referrers. The two carry the same record, defined once in Section 8.3.

Table 12: Where each authenticity object lives, and why it cannot live elsewhere.

Object	Where it lives	Why not elsewhere
Trust root	Inside the snapshot document, which is the config blob of a published brain.	As a module or a layer it could be omitted by a selective install, leaving a consumer with nothing to verify against.
Signatures	Beside the snapshot in an export; mirrored as OCI referrer artifacts on publication.	As a layer, countersigning would change the brain's digest; as referrers only, they would not survive leaving a registry.
Pinned digest	In consumer-side state, never in the artifact.	A pin an artifact could supply would be a pin the attacker supplies.

8.9 How verification runs

A consumer performs two independent checks and MUST report them separately rather than as one boolean.

1. **Integrity, bottom-up.** Recompute block identifiers from stored bytes, recompute each module's root from its composition, and confirm the snapshot names those roots. This requires nothing but the bytes, and its result MUST NOT depend on any trust configuration.
2. **Authenticity, top-down.** Verify a signature over the snapshot's canonical bytes under the protocol namespace; confirm the key inside the signature is present in the trust root in force at that position in the chain; confirm the key holds every scope the snapshot's difference from its first parent required; confirm the trust root's digest matches the pin, if one exists; and confirm that the key is neither retired nor compromised at that position.

Keeping them distinct matters in practice. A consumer with no trust anchor can still verify integrity completely, which is the zero-configuration case that must keep working. And a consumer that reports a single verified flag cannot express "intact, and signed by an authorized key" as distinct from "intact, provenance unknown", which is the distinction Section 11 carries into the Evidence Bundle.

Verification policy is configuration. As with query planning and retention, the protocol fixes the checks and the reporting, not the deployment's appetite for risk. A consumer-side *verification policy* states whether an unsigned brain may be installed (default: warn and permit for a first pull, refuse for a brain previously seen signed), how many valid signatures a head must carry (default: one), whether a `propose`-scoped head may be treated as the current state (default: no), and which trust-root digests are pinned. What is not configurable is whether the result is reported: an implementation MUST NOT present an unverified brain as verified, whatever the policy allowed it to install.

8.10 Three cases, worked through

The rules above are short, and their consequences are not obvious until they are run against a situation. Three cases cover what a deployment actually meets: admitting an owner, a key admitting itself, and starting from nothing.

In the first two, the brain is at revision 1 with keys A and B, both holding `govern`, a `govern_quorum` of 2, and a chain reaching `S7`. A consumer has pinned the digest of revision 1.

Case 1: admitting a third owner. C generates a key pair and sends the *public* half. The channel need not be confidential, since nothing secret is transmitted, but the recipients do need independent grounds to believe the key is C's, which is an out-of-band question the protocol does not answer. A trust root at revision 2 is built listing A, B, and C; a snapshot `S8` is built naming `S7` as its parent, carrying that trust root, and leaving every module root equal to `S7`'s, because a revision changes nothing else (Section 8.4). A and B each sign `S8`, producing two detached signatures over the same bytes.

When C later commits `S9` and signs it, a consumer verifying that snapshot resolves a chain of five questions, and the last one is where it terminates:

1. Does the structure recompute? Integrity, requiring no trust (Section 8.1).
2. Does C's signature verify over `S9`'s canonical bytes?
3. Which trust root is in force at `S9`? The one its first parent names, which is revision 2, and C is listed there.
4. Which scope did `S9` require? Its difference from `S8` shows a non-canonical composition that gained blocks, so `commit`, which C holds.
5. Why is revision 2 itself trustworthy? Because `S8` introduced it carrying two signatures from A and B, who held `govern` in revision 1, whose digest matches the pin.

Nothing in this requires C to do anything unusual: C commits and signs normally. What makes those signatures carry weight is that an earlier snapshot in the same chain authorized the key, signed by parties who were already authorized. The authorization travels inside the brain, so it reaches every consumer without anyone fetching anything extra.

Case 2: a key admitting itself. C installs the brain, which is public and requires no permission, edits the trust root to include their own key, and builds `S8`′ naming `S7` as its parent. C signs it with their own key.

The first two checks pass, and it is worth being explicit that they do. The structure recomputes, because C recomputed it correctly. The signature verifies, because C genuinely produced it. *Integrity does not detect this attack*, which is the whole reason Section 8 exists.

The third and fourth checks fail:

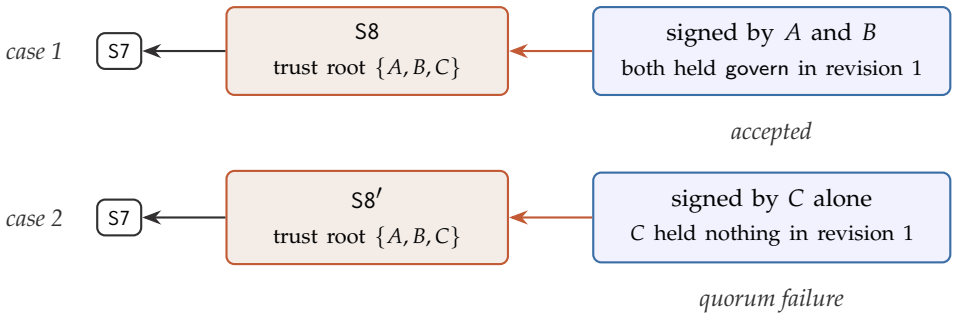

Figure 9: The same key list, admitted two ways. Both snapshots recompute correctly and both carry a signature that verifies, so integrity separates them not at all. What separates them is whether the snapshot introducing the new list carries approvals from keys that were already authorized, which an attacker cannot produce without the corresponding private keys.

- The trust root in force at `S8′` is the one its parent `S7` names, which is revision 1, holding only *A* and *B*. *C*'s key is absent: *unauthorized key*.
- `S8′` changes the trust root relative to its first parent, so it is a revision, and requires two signatures from keys holding `govern` in revision 1. It carries none: *quorum failure*.

The comparison with Case 1 is the point of stating both, and it is sharper than it first appears (Figure 9):

> The key list in Case 1 and the key list in Case 2 may be **byte-identical**. The same three keys, the same scopes, the same serialization. What distinguishes them is not the list. It is *who signed the snapshot that introduced it*.

An attacker can write any key list they like. What they cannot write are its approvals, because producing one requires a private key they do not hold. Two further consequences follow. Rejection propagates forward: a later snapshot naming `S8′` as its parent draws its trust root from a snapshot already rejected, so the whole subtree falls with it. And this case fails *without any pin at all*, provided the consumer holds the chain, because the verifier is checking an internal transition rule rather than comparing against an external anchor. The chain the attacker must ship contains the evidence that the attacker's own change of authority was never approved.

Case 3: starting from nothing. There is no prior state, so nothing can be derived and the first authority is declared. *A* and *B* generate their key pairs; a trust root at revision 1 lists both with a `govern_quorum` of 2; a genesis snapshot is built with an empty `parents` list, that trust root, and empty module compositions; and both keys sign it, satisfying the quorum the trust root itself declares (Section 8.6).

Read from inside the artifact, this proves nothing, and the asymmetry with the first two cases is the useful thing to notice. There, what decided the outcome was a rule a verifier could check against material it already held. Here there is no such material: the trust root asserts that a key is authorized, and the entity asserting it is that key. Any party can construct an indistinguishable object.

What gives a genesis meaning is therefore the step that happens outside it: the digest of revision 1 is published through an independent channel and pinned by consumers on first contact (Section 8.7). From that moment the derivable rules take over, and Cases 1 and 2 behave as described. Stated compactly: *a genesis is not validated, it is anchored*, and it is the only object in this protocol of which that is true.

8.11 Which impersonation routes remain closed

The design is worth judging against the attacks it is meant to stop, including the two it does not (Table 13). The mechanism that closes most of them is the same one throughout: a signature is a value only the holder of a private key can produce, and a trust-root revision is not a claim about the key list but a document that must carry the signatures of keys authorized in the previous list. An attacker can write any key list they like; they cannot write its approvals.

Table 13: Impersonation routes against a signed brain. The last two rows are honest limitations rather than defences.

Route	Outcome
Add blocks, recompute roots, sign with the attacker's own key.	Rejected: the key is absent from the trust root in force.
Add blocks and strip the signatures.	Rejected for any consumer that has seen this brain signed; reported as unsigned rather than as valid.
Add blocks and ship an attacker-authored trust root listing their key.	Rejected: the trust root's digest does not match the pinned one.
Add the attacker's key to the existing trust root.	Rejected: the revision requires `govern` signatures from keys authorized in the previous revision, and the attacker holds none of their private keys.
Reuse a legitimate signature over modified content.	Rejected: the signature covers the snapshot's canonical bytes, and modified content is a different snapshot.
Recombine legitimately signed module roots into a state that never existed.	Rejected: signatures cover snapshots, not roots, and no signed snapshot names that combination.
Backdate a signature made with a stolen key.	Rejected: validity is positional, so a `compromised_from` record withdraws trust by chain position and a written timestamp decides nothing.
Serve an older, legitimately signed snapshot indefinitely.	*Not prevented.* Withholding is a freshness problem, not an authenticity one (Section 15).
Present a fabricated brain to a consumer who has never seen the genuine one.	*Not prevented.* First contact is irreducible (Section 8.7).

8.12 Two deployments, one format

Nothing above changes with the size of a deployment; only its configuration does. Two cases are worth stating because they are the common ones and because they exercise opposite ends of the same structure.

A brain with a single author. One key, holding every scope, with `govern_quorum` of 1. The trust root has one entry, signing is one command at publication, and the value obtained is not access control but tamper-evidence: a brain republished by anyone else fails verification for every consumer who has pinned this one. The single author does not need a second key to benefit, and adding a second later is an ordinary trust-root revision signed by the first.

Public read, restricted write. Anyone may install the brain; a few may publish it. Reading is a registry concern and requires nothing from this section, and the trust root governs whose authorship a consumer *accepts* rather than who is physically able to write. Maintainers hold `commit`, a subset holds `govern`, canonical drops are restricted to keys holding `drop:canonical`, and outside contributors hold `propose`, which is enough for their snapshots to be attributable and not enough for one to be mistaken for the published state. Section 13 takes that arrangement and defines what happens when a contribution actually arrives.

8.13 What this deliberately does not solve

With the above in place, a hostile registry cannot fabricate state: everything verifies offline against the pinned trust root. It can still *withhold*, serving an older, legitimately signed snapshot indefinitely while concealing that newer ones exist. Signing does not address this, and claiming otherwise would be misleading. It is treated as a stated limitation in Section 15 and named again in Section 17.

9 The Boltzmann Protocol

An installed OCI Artifact is passive data. The *Boltzmann Protocol* is the contract through which any conforming client interacts with a brain, along two paths, query and ingestion. Boltzmann Brain is therefore delivered as a protocol rather than as a deployable service or container image: the brain is portable data, and any client that speaks the protocol can read and extend it. The protocol includes no LLM of its own; it governs structure, identity, validation, indices, snapshots, and distribution.

The operations divide into five *roles*. The division is not decorative: a role is the unit of conformance (Section 16) and the unit of capability. A brain published for consumption needs only the reading role implemented on the consumer's side; a client that never writes need not implement validation at all; and a consumer that recomputes integrity while holding no trust anchor need not implement authenticity, which is exactly the zero-configuration case Section 8.1 guarantees. Table 14 states the surface.

Because these are protocol operations rather than a bundled engine, they can be implemented independently and in any language, and the brain never depends on a particular process being kept running. `pack` in particular is defined to produce the same on-disk layout that a registry would serve, so that any OCI tool can copy the result without implementing this protocol at all.

Protocol version. Every block envelope, composition document, snapshot, and published manifest declares the protocol version it conforms to. A client that meets a document declaring a version it does not implement MUST refuse it and say so. This is a coarser check than schema versioning (Section 6.9) and answers a different question: not "can I read this knowledge" but "is this a Boltzmann artifact at all".

Failures are typed. An implementation MUST distinguish, and report distinguishably, at least the conditions in Table 15. Collapsing them into a generic error defeats guarantees stated elsewhere in this document: a consumer that cannot tell a redacted block from a corrupted one has lost the property Section 12.5 requires, and one that cannot tell an uninstalled module from an empty one will silently answer queries from a subset of the brain it thinks it has.

9.1 The boundary between the protocol and the external LLM

The protocol defines which memory types exist and which schema each block must satisfy. The external LLM performs the semantic operations: reading, interpreting, extracting concepts,

Table 14: Protocol operations by role. Every operation is defined over an installed snapshot; none of them requires a running service.

Role	Operation	What it does
Reader	`snapshot`	Report the current snapshot and its installed modules.
	`root_of`	Report the Merkle root of one installed module.
	`open_index`	Open an index of a named kind over a module.
	`resolve`	Resolve a `block_id` to its bytes, verifying the hash.
	`prove`	Produce an inclusion proof of a block in a module.
	`resolvability`	Report which blocks are resolvable and which are tombstoned.
	`verify`	Recompute the whole hash structure of an installed snapshot (Section 8.9).
	`browse`	Walk an axis of the catalog and list what is filed under a class (Section 6.7).
	`search`	Answer a query with an Evidence Bundle (Section 11).
Authenticity	`sign`	Produce a detached signature over a snapshot under the protocol namespace.
	`authenticate`	Check signatures against the trust root in force at a snapshot's position.
	`pin`	Record a trust root digest as the anchor for this brain.
	`rotate`	Commit a trust-root revision, under the quorum rule.
	`revoke`	Record that a key is retired, or compromised from a chain position.
Writer	`init`	Create a brain: a genesis snapshot with no parents and its first trust root (Section 8.6).
	`register`	Preserve a source as canonical evidence, with provenance.
	`replace`	Register a new edition and record supersession.
	`define_task`	Emit a processing task for an external model.
	`validate`	Judge candidate blocks against the protocol's checks.
	`classify`	File canonical blocks under classes on a declared axis (Section 6.7).
	`commit`	Incorporate validated candidates and advance the snapshot.
	`merge`	Reconcile two histories into a snapshot naming both as parents.
	`rebase`	Replay a history onto another, minting new snapshot identities.
	`squash`	Collapse a run of snapshots into one, preserving provenance.
Retention	`drop`	Exclude blocks from a module, cascading through provenance.
	`drop_by_producer`	Exclude everything a given model, pipeline, batch, or actor produced.
	`supersede`	Record that a newer block takes precedence.
	`demote`	Reduce a block's retrieval priority without removing it.
	`prune`	Reclaim blocks no retained root still references.
	`redact`	Destroy bytes a retained root still names, under policy.
Distribution	`pack`	Materialize the current snapshot as an OCI Artifact locally.
	`push`	Publish, refusing to overwrite a diverged remote (Section 7.4).
	`pull`	Install an artifact, optionally only some modules.
	`fetch`	Retrieve a remote history without moving the local pointer (Section 13.6).

Table 15: Failure conditions a conforming implementation must distinguish, grouped by what a caller can do about them.

Condition	Meaning
Identity and integrity	
Unknown schema	The block's schema version is not implemented here.
Block not found	The identity is not held by this brain.
Block tombstoned	The identity is known; the bytes were destroyed by redaction.
Integrity failure	Stored bytes do not hash to the identity they claim.
Inclusion proof failure	A membership proof does not reconstruct the root.
Membership failure	A returned block is not in the installed snapshot.
Scope of the installed brain	
Module not installed	The snapshot does not name this module.
Stale index	A travelling index is bound to a root other than the installed one.
Append-only violation	A drop was attempted on the episodic module.
Policy refusal	The retention policy does not authorize this removal.
Authenticity (Section 8)	
Unsigned brain	A signature was required and none is present.
Signature invalid	The signature does not verify over the snapshot's canonical bytes, or was made under another namespace.
Unauthorized key	The signing key is absent from the trust root in force at this position.
Insufficient scope	The key is authorized, but not for the change this snapshot made.
Retired key	The key was authorized once and not at this position; earlier signatures by it stand.
Compromised key	The key's signatures are withdrawn from a recorded position onward.
Quorum failure	A trust-root revision lacks the required `govern` signatures from the previous revision.
Trust root mismatch	The trust root does not match the pinned digest.
Reconciliation (Section 13)	
Divergence	The remote is not an ancestor of the local snapshot.
No common ancestor	Two histories share no ancestor, so no three-way merge is defined.
Governance conflict	The histories being reconciled carry different trust roots.

recognizing episodes, proposing procedures, and contextualizing results. *The LLM proposes; the protocol governs what is stored.*

This boundary keeps the brain provider-independent. The same snapshot can be used with a commercial service, a development agent, or a local model, as long as the client implements the Boltzmann Protocol. Trust does not rest on a privileged service: because every block, snapshot, and module is addressed by its hash, any client can verify integrity independently of who produced it.

Design rule

The LLM never writes directly to the Merkle DAGs or to the indices.

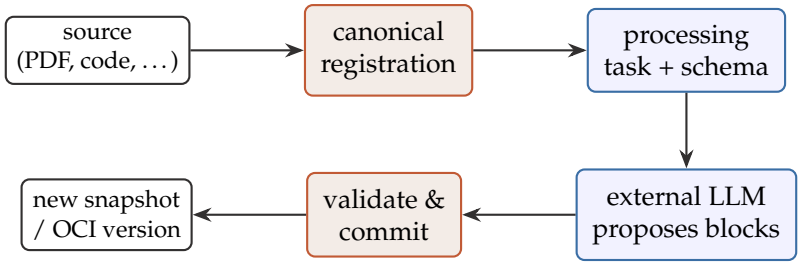

Figure 10: The brain conserves and validates; the external LLM interprets and proposes blocks. Only validated candidates are committed and versioned.

10 Ingestion via an External LLM

Ingestion separates the preservation of a source from its interpretation (Figure 10). First the original is registered deterministically; then the protocol delegates semantic processing to the user's LLM.

10.1 Canonical registration

Canonical evidence enters through three protocol operations. There is no in-place edit of a source: a new edition is always a new block.

Register.

1. The user provides a PDF, text, repository, conversation, image, or other source.
2. The LLM invokes the `brain.ingest` operation through the available skill or adapter.
3. The format is validated, its SHA-256 computed, and exact duplicates detected; re-registering an identical blob is a no-op.
4. The original is stored as an immutable canonical block, optionally together with a normalized view produced by a named deterministic pipeline.
5. The protocol records who incorporated it, when, from where, and under what license or policy.
6. The canonical Merkle root changes and a new snapshot becomes available, even though no semantic interpretation exists yet.

Registering a source does not declare it true. The canonical module asserts that the evidence was incorporated and preserved.

Replace. When a newer edition of a source should take precedence, the protocol registers the new original as a distinct block and records a `supersedes` relation to the previous one. That relation is a provenance edge rather than a field of either block, so both remain pure statements about the bytes they preserve and the precedence between them stays auditable where every other actor-dependent record lives. The caller chooses whether the superseded original remains in the current composition (kept for audit) or is dropped in the same commit. In either case the new block is what subsequent processing tasks read. `replace` is therefore register plus a supersession edge, with an optional drop of the old evidence, not a mutation of bytes already stored.

Drop. Excluding a canonical block from the current composition is the same drop operation defined in Section 12, with a privileged cascade: every derived block that cites the dropped evidence is dropped by default in the same commit, unless the caller has already registered a replacement and requests re-derivation against it. Dropping canonical evidence is allowed only under explicit policy (Principle 2).

10.2 Processing task

After canonical registration, the protocol returns a structured task to the LLM. The task identifies the source, the instructions, the allowed memory types, the required references, and the exact output schema:

```
{
  "operation":            "extract_knowledge",
  "source":               "sha256:PDF123",
  "allowed_memory_types": ["episodic", "semantic", "procedural"],
  "requirements":         ["cite source ranges", "do not invent"],
  "output_schema":        "boltzmann.candidates/v1",
  "task_id":              "ingest-2026-07-24-01",
  "instructions":         "..."
}
```

The LLM reads the source and returns typed candidate blocks. A document may produce semantic and procedural memory without producing episodic memory; the classification depends on what the source actually contains.

Two operations are defined. `extract_knowledge` asks the model to read a registered source and propose typed blocks from it. `rederive` asks it to rebuild derived blocks against a replacement canonical source, which is the operation a caller requests when dropping evidence that has since been superseded (Section 12.3).

The `task_id` is what the resulting provenance records cite, so that a later batch invalidation can name this task as the producer of everything it yielded.

Only three memory types are proposable

A processing task MUST NOT allow candidates in the **canonical** or **provenance** modules, and an implementation MUST reject a task that does. Canonical registration is deterministic: it preserves observed bytes and needs no interpretation. Provenance is written by the protocol itself. Opening either to a proposer would put the external model in charge of the evidence or of the audit record, which is precisely the boundary of Section 9.

Wire schemas. Three schema identifiers are normative, and each is carried explicitly in the document it governs so that a receiver never has to infer it: `boltzmann.task/v1` for the processing task, `boltzmann.candidates/v1` for the model's response, and `boltzmann.evidence/v1` for the Evidence Bundle (Section 11).

10.3 Validation and commit

Results from the LLM do not enter automatically. Validation is the point at which a *proposal* becomes a *block*: before it, a candidate is untrusted data with no identity; after it, an accepted candidate has a `block_id` and is eligible for commit. No other operation may mint a block.

Every candidate receives exactly one of four verdicts (Table 16), and only `VALIDATED` candidates MAY be committed. A verdict MUST be accompanied by the issues that produced it,

each naming the check that fired and, where applicable, the field within the payload, so that a rejection is actionable.

Table 16: The four validation verdicts.

Verdict	Meaning
`VALIDATED`	Well-formed, referenced, and consistent. Eligible for commit.
`PENDING_REVIEW`	Admissible, but not decidable by the protocol alone. Requires a human or a policy decision.
`REJECTED`	Malformed, unreferenced, or duplicate. Never committed.
`CONTRADICTED`	Well-formed but in conflict with knowledge already held; the conflicting blocks are named.

The checks. A conforming implementation MUST apply at least the following. *Schema*: the payload satisfies a registered schema for its declared memory type. *Allowed type*: the candidate's memory type was permitted by the task. *Evidence*: every cited canonical block exists in the installed snapshot. *Evidence consistency*: the citations a candidate states match the source the task named, since a payload claiming different evidence is a validation failure and not something to reconcile silently. *Content*: bytes a candidate names are present and hash as claimed. *Duplicate*: the resulting identity is not already in the composition. *Relation*: referenced blocks exist and the relation is admissible. *Contradiction*: the candidate does not conflict with knowledge already held.

How contradiction is detected is left open, and a check that cannot decide MUST yield `PENDING_REVIEW` rather than guess in either direction. The protocol fixes what the verdicts mean and which of them may be committed, not how cleverly an implementation reaches them.

For each accepted block, the protocol prescribes the following steps:

1. Serializes the block canonically.
2. Computes its `block_id` by hashing.
3. Stores the block as an immutable object.
4. Connects it to the source via provenance.
5. Incorporates it into the episodic, semantic, or procedural Merkle DAG.
6. Updates B^+ trees, lexical, vector, and graph indices as appropriate.
7. Creates a new snapshot and, when published, a new OCI version.

11 Query via an External LLM

For now, Boltzmann Brain is conceived as a medium for LLMs. The user converses with their usual LLM; a skill or adapter lets that LLM query the brain through the protocol. The brain returns raw or semi-raw knowledge, and the LLM turns it into an answer appropriate for the task (Figure 11).

11.1 Query flow

1. The user asks the external LLM a question.
2. The LLM interprets the intent and calls `brain.search` with the query and optional filters.

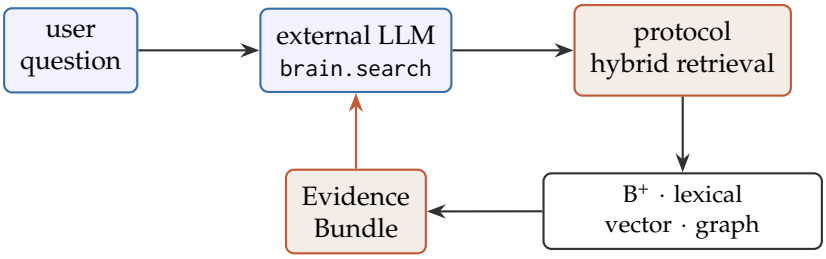

Figure 11: The external LLM queries the brain and contextualizes the returned Evidence Bundle. The protocol combines indices, verifies hashes, and returns data rather than prose.

3. The protocol combines B^+ trees, lexical search, vectors, and graph.
4. Knowledge blocks and their canonical sources are retrieved.
5. The protocol verifies hashes and membership in the installed snapshot.
6. An Evidence Bundle is returned.
7. The LLM explains, summarizes, compares, or uses the evidence.

11.2 Query planning and guarantees

Step three above hides a query planner. By Principle 7 a query is declarative: the caller expresses intent (a query, optional filters over memory type, class, or recency, and optional hints such as a retrieval mode or a result limit) and never names a physical index. Choosing which indices to use, and how to combine them, is the responsibility of a conforming implementation, in the same way that the SQL standard defines a query language and its semantics while each database ships its own optimizer. Selection follows the shape of the query: exact identifiers resolve through the hash map, ordered or range predicates through the B^+ tree, term matches through the inverted index, natural-language intent through the vector index, neighborhood and traversal through the graph, and categorical filters through bitmaps. In practice retrieval is hybrid: filters narrow a candidate set, lexical and vector search generate candidates in parallel, their rankings are fused, the graph expands from the strongest hits, and the hash map resolves and verifies each result.

The protocol fixes the contract and the invariants, not the algorithm. A conforming implementation must return knowledge blocks with their provenance and a retrieval score, never prose; every returned block must be verified against the installed snapshot by hash and membership; and no single index may be treated as authoritative. The fusion method, ranking weights, graph expansion depth, and any adaptive intent classification are deliberately left to the implementation. A consequence worth stating explicitly is that the protocol guarantees *verifiability, not identical ranking*: two conforming implementations may return the same verifiable set of blocks in a different order, because vector search is approximate and ranking is tunable.

The request. A query separates three things: the terms, the conditions that narrow the snapshot, and the advice a planner may follow or ignore.

```
{
  "text": "how do I compute Fourier coefficients",
  "filters": {
    "memory_types":        ["semantic", "procedural"],
    "classes":             ["sha256:CLASS_FOURIER"],
    "since":               "2026-05-01T00:00:00Z",
```

```
    "until":              null,
    "tags":               ["exam"],
    "evidence":           ["sha256:PDF123"],
    "include_superseded": false
  },
  "hints": { "mode": "auto", "limit": 10, "expand_depth": 1 }
}
```

The `text` MAY be empty. "The episodes of last May" is a complete request with no terms in it, and refusing it would make recency and class filters unusable on their own.

Filters are conditions and MUST be honored. Hints are advice: the retrieval mode, the result limit, and the expansion depth express what the caller believes will work, and an implementation that ignores every hint and still returns verified blocks with their provenance is conforming. The named modes are `auto` (infer intent from the query's shape), `exact` (resolve identities only), `lexical` (match terms as written), `semantic` (match meaning, accepting approximation), and `associative` (expand outward from the strongest hits along declared relations). A mode names a strategy, never an engine, which is what keeps Principle 7 intact.

The `classes` filter names class blocks (Section 6.7), which is what makes "the past exams on Fourier" a condition the protocol can honour instead of free text a responder may read as it likes. The `memory_types` filter is the mechanism behind R2: it is what keeps "what happened in the class of May 14" from competing with "the definition of a Fourier series" in a single similarity ranking. The `include_superseded` filter reflects that superseded blocks stay in the composition and remain verifiable; what supersession changes is accessibility (Section 12).

11.3 Evidence Bundle

The brain's result is not a written answer. It is a data contract carrying the matching blocks and everything needed to audit them:

```
{
  "matches": [{
    "block_id":    "sha256:CONCEPT1",
    "memory_type": "semantic",
    "content":     { ... the block's payload ... },
    "score":       "0.87",
    "sources":     [{"block_id": "sha256:PDF123",
                     "locator":  "p.147",
                     "citation": "verified"}],
    "verified":     true,
    "resolvable":   true,
    "superseded_by": null
  }],
  "verified_against": { "semantic": "sha256:SEMANTIC_ROOT_V7" },
  "authorship": {
    "state":      "authorized",
    "snapshot":   "sha256:SNAPSHOT",
    "key":        "SHA256:5tS7wqf2K...",
    "trust_root": "sha256:TRUST_ROOT_R3",
    "pinned":     true
  },
  "truncated": false
}
```

Six fields carry guarantees stated elsewhere in this document, and a conforming implementation MUST populate all of them.

`verified` asserts that both the block's hash and its membership in the installed snapshot were checked. `verified_against` names the module roots that membership was checked against, so the assertion is re-checkable independently rather than taken on the responder's word: without it, `verified` would be a claim a consumer has no way to audit, which is the opposite of the property Section 4.3 argues does not compose.

`resolvable` distinguishes a block whose bytes are still present from one that was redacted. A consumer MUST be able to tell a removed block from a corrupted one (Section 12.5), and this is where that requirement is met at query time.

`superseded_by` names a newer block that takes precedence, so a caller that asked to include superseded blocks can tell which ones they are.

`citation` states what became of each cited source, and it exists because verification does not otherwise survive selective installation. A bundle's sources point at canonical blocks, and under selective installation the canonical module may not be installed at all (Section 7.2), so a consumer can receive a fully verified semantic block citing evidence it has no way to check. Since R1 makes canonical evidence the root of auditability, the bundle MUST say which of three situations holds: `verified`, the cited block is present in the installed canonical composition; `uninstalled`, the canonical module is not installed here and the citation could not be checked; or `missing`, the canonical module *is* installed and the citation is not in it, which is a violation of R1 and MUST be reported rather than smoothed over. Collapsing the last two would hide a broken brain behind an incomplete install.

`authorship` reports the second of the two checks of Section 8.9, separately from `verified`, which reports the first. Its `state` is `authorized` when a signature over the installed snapshot verified under a key holding the scopes that snapshot's change required, `unsigned` when the brain carries no signature at all, and `unauthorized` when a signature was present and failed any of those conditions. A conforming implementation MUST NOT fold authorship into `verified`: a bundle that collapses the two cannot express "intact, and signed by an authorized key" as distinct from "intact, provenance unknown", and a caller has no way to recover the difference afterwards.

`score` is a decimal *string*, for the same reason payloads forbid floats (Section 6.2): a number whose textual form varies across languages does not belong in a wire format meant to be compared. The scale is not normative; only the encoding is.

The consumer receives block identities, content, memory type, sources, and verification status, and remains free to explain, summarize, compare, or reuse the evidence. Those transformations happen outside the brain.

12 Retention, Forgetting, and Erasure

Everything described so far only adds. Content addressing and immutable blocks make individual units append-only, but a module version is a *composition* of those units, and compositions can exclude what should no longer belong. Without an explicit way to drop blocks, a brain that only grows is neither practical nor, in some jurisdictions, lawful, and it cannot recover from knowledge that was deliberately wrong.

12.1 Why exclusion must be first-class

Three pressures call for different responses, and they should not be conflated.

Wrong or obsolete knowledge. A semantic definition may be incorrect, a procedure may be outdated, or a canonical source may have been ingested in error. Leaving the block in the

current composition and merely down-ranking it is not enough: the wrong claim still lives in the snapshot that consumers install. The protocol needs a way to *exclude* it from the next version.

Cost and signal. Unbounded growth costs storage and precision. Stale near-duplicates compete for the top of every ranking, so a brain that never excludes gets measurably worse at answering.

Law and safety. Personal data, credentials, or licensed material may have to disappear even from retained history [9]. That case is stricter than exclusion: it requires destroying bytes that a retained root still names. It is treated separately in Section 12.5.

12.2 What biology suggests

Forgetting in nervous systems is a regulated process, not a storage failure. Microglial pruning eliminates connections that are not reinforced [30], active forgetting is driven by dedicated molecular machinery [8], and engram studies treat inaccessibility as adaptive plasticity rather than erasure [35]. Cognitive psychology separates the availability of a trace from its accessibility [3]. The architectural reading is that exclusion from the current composition is the default, that reclamation of what no retained version needs is a background process, and that destroying evidence is a rare, policy-bound exception rather than the ordinary cleanup path.

12.3 Drop: excluding blocks from a module

The primary removal operation is *drop*. It applies to the **canonical**, **semantic**, **procedural**, and **provenance** modules. The **episodic** module is exempt: episodes are a chronological record of what happened, so corrections append new episodes or supersession relations (Section 5).

A drop does not mutate blocks in place. Blocks remain content-addressed and immutable. What changes is the *composition* of the module: the protocol builds a new Merkle DAG whose leaves are the surviving `block_ids`, computes a new root, rebuilds the module's indices from that composition, and records the removal in provenance (Figure 12). The dropped blocks become unreachable from the new root and are eligible for pruning once no retained root references them. Structural sharing still applies: unchanged blocks are reused by hash, so the cost of a drop is proportional to what was removed, not to the size of the module (Section 6).

Example

`semantic-root v1` contains `K1`, `K2`, and `K3`, where `K2` is a wrong Fourier definition. A drop of `K2` yields `semantic-root v2` over `K1` and `K3` only. Indices are rebuilt without `K2`, and provenance records that `K2` was dropped, by whom, and why. Optionally the external LLM re-derives a corrected $K2'$ from the canonical source and a later commit adds it under `v3`. The wrong block never appears in `v2` or `v3`.

Provenance cascade. Dropping a block is not always local, and the cascade differs by module.

Dropping a *semantic* or *procedural* block removes it from that module's composition and rewrites or removes provenance edges that pointed to it. Downstream dependents are treated the same way. The canonical source remains, so the caller may later re-derive a corrected block from the same evidence.

Dropping a *canonical* block is privileged. Because derived knowledge cites that evidence as its root, the protocol always walks the dependency closure and, by default, *drops* every semantic and procedural block that listed the canonical as evidence, in the same commit. Re-derivation is

not the default: it runs only if the caller has already registered a replacement canonical (`replace`) or explicitly requests `rederive` against another canonical still in the composition. Each affected module receives a new Merkle root, so a single logical removal of evidence can publish several new module versions at once. Provenance records the actor, the policy, the reason, and the resulting roots, so the removal stays auditable even though the dropped content is no longer part of the current composition.

Example

The wrong lecture PDF was ingested as `PDF_wrong`, and several semantic blocks cite it. A `drop` of `PDF_wrong` yields a new canonical root without that blob; the cascade drops the derived definitions in the same commit. Registering the correct PDF and running a processing task then produces new candidates under a later snapshot. The wrong source never appears in the current composition, and neither do interpretations that depended only on it.

The cascade does not weigh evidence, and cannot. The cascade excludes every derived block that cited the dropped evidence, without distinguishing a block supported by one source from a block supported by three. The obvious refinement, dropping a derived block only when it loses *all* of its evidence, is not implementable here, and the reason is worth recording because the refinement looks reasonable until it is attempted. Citations live in the payload, and therefore inside `block_id` (Section 6.1). A semantic block citing three sources that loses one cannot "keep the other two": a block with two citations is a different block, with a different identity. A block whose citation set no longer holds cannot stay, whatever fraction of that set survived.

What remains open is narrower than it first appears. Rebuilding such a block against its surviving evidence currently requires a full round trip through the external model, even though the surviving evidence has not changed. Whether the protocol should offer a cheaper deterministic path for that case is stated as open in Section 17.

Batch invalidation. Because provenance records the producer of each derived block (model, pipeline, ingestion batch, or actor), a drop may be stated over a *set*: everything produced by a given ingestion, or everything derived by a given model version. That is the natural response to deliberately wrong knowledge introduced in bulk, and it reuses the same cascade rather than inventing a second mechanism.

12.4 Companion mechanisms

Drop is the ordinary way to remove knowledge from a non-episodic module. Three companion operations cover accessibility, reclamation, and the rare case where bytes must disappear while a retained root still names them (Table 17).

Supersession and demotion. A block is marked as superseded by a newer one, or its retrieval priority decays. It stays in the current composition and remains verifiable; what changes is accessibility. This is the default path for the episodic module, and an optional soft path elsewhere when history inside the current snapshot must remain queryable under an explicit filter.

Pruning of unreachable blocks. A brain retains a set of roots (tagged releases plus recent snapshots). After drops, former leaves are unreachable from those roots and can be reclaimed by mark-and-sweep [5]. Pruning never decides *what* to forget; it reclaims what no retained composition still needs.

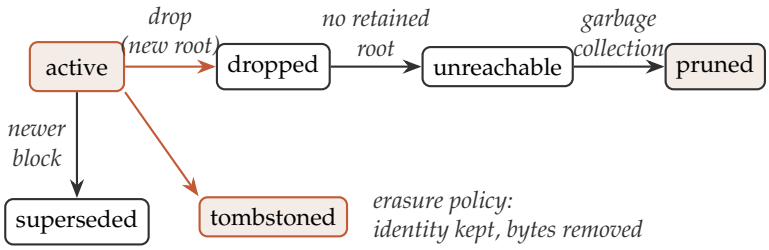

Figure 12: Lifecycle states of a knowledge block. The ordinary removal path for non-episodic modules is drop: a new Merkle root excludes the block, provenance is updated, and pruning later reclaims bytes once no retained root references it. Supersession keeps the block in the composition; redaction destroys bytes that a retained root still names.

Redaction of referenced content. Only when content that a retained root still references must be destroyed does the protocol remove bytes: a *tombstone* deletes the bytes and leaves an auditable record; *crypto-shredding* destroys the encryption key [14]; a *lineage rewrite* rebuilds history so the block was never referenced. Redaction is the path for destroying retained canonical bytes under law or safety policy. It is not the cleanup path for a wrongly ingested source or for wrong semantic knowledge: those are dropped.

Append-only is not an exemption from erasure. The episodic module is exempt from drop, and the law-and-safety pressure named earlier in this section can require personal data to disappear. Those two facts meet in an uncomfortable place, and this document says so plainly: for personal data inside an episode, redaction is the *only* available path, so the mechanism framed here as rare and exceptional is the ordinary one for the module most likely to contain personal data. Three consequences are normative. A retention policy MUST be able to declare episodic content redactable (Table 18). An implementation MUST NOT read the append-only rule as an exemption from an erasure obligation, since the two constrain different things: append-only governs how a record is *corrected*, not whether its bytes may be destroyed. And a deployment recording episodes about identifiable people SHOULD decide in advance which episodic content is redactable, rather than meet the question for the first time when a request arrives.

12.5 Why drop preserves a clean snapshot

A drop publishes a new composition. Consumers of the new root never see the dropped block: membership proofs, indices, and Evidence Bundles are computed over the survivors only. Older roots, if still retained, continue to verify exactly as before, because nothing about them changed. The brain therefore offers a *clean current snapshot* after every drop, which is the property that makes exclusion usable for wrong knowledge.

Redaction is different: it punches a hole in a composition that still names the block. The Merkle DAG references identities, not bytes, so deleting the bytes behind a leaf changes no hash and membership still verifies, but reconstruction of that one block is forfeited. A conforming implementation must report which blocks of a snapshot are resolvable and which are tombstoned, so that a removed block is never indistinguishable from a corrupted one. Provenance is again the home for that record.

Two limitations of redaction deserve stating. A hash of low-entropy content is not anonymous, so confirming a guess may still be possible when the `block_id` is kept. And erasure does not propagate by itself across already-pulled copies: the protocol can publish a revocation, but a distributed brain can only signal destruction, not guarantee it. Neither limitation applies to

Table 17: Removal mechanisms and their guarantees.

Mechanism	What it removes	Reversible	Effect on current roots
Drop (non-episodic)	Membership in the module composition.	Via a later commit that re-adds.	New Merkle root without the block; indices rebuilt; provenance updated. Canonical drops always cascade to dependents.
Supersession / demotion	Accessibility, not membership.	Yes	None; block stays in the composition.
Pruning	Bytes of blocks in no retained root.	No	None; retained roots unaffected.
Redaction (retained root still names the block):			
Tombstone	The bytes; identity kept.	No	Root intact; block unresolvable.
Crypto-shredding	The plaintext, via the key.	No	Root intact; ciphertext still verifies.
Lineage rewrite	The block and its history.	No	Prior roots invalidated; new lineage published.

an ordinary drop of wrong semantic or procedural knowledge, because the new root simply never included the block.

12.6 Policy is configuration

As with query planning (Section 11), the protocol fixes the operations and the invariants, not the thresholds. What the protocol requires is that every drop, supersession, prune, and redaction be explicit, recorded in provenance, and reportable, which is the content of Principle 8. What a deployment decides is stated in a *retention policy* (Table 18), which an implementation MUST consult before performing any removal and MUST refuse to act against.

Three of these defaults are deliberately restrictive. Canonical drops are forbidden by default because excluding evidence forfeits re-derivation from that source, which Principle 2 makes a privileged act. Both redaction settings are forbidden by default because redaction is the mechanism for law and safety, not the cleanup path, and a deployment that has not decided which content is redactable has not decided that it is; the episodic setting is separate precisely because it is the module where the question is most likely to arise and least likely to have

Table 18: The retention policy: what a deployment decides, and the defaults this version specifies.

Setting	Decides	Default
Droppable modules	Which modules permit `drop`.	all non-episodic
Canonical drop	Whether canonical evidence may be excluded at all.	forbidden
Cascade review	How many dependents a cascade may drop before the commit requires review.	no threshold
Retained roots	How many recent snapshots stay reachable, on top of tagged releases.	10
Redactable content	Which media types may have their bytes destroyed.	none
Episodic redaction	Whether episodic content may be redacted, which is the only erasure path an append-only module has.	forbidden
Allowed mechanisms	Which removal mechanisms are permitted at all.	all the module allows

been considered. The episodic module can never appear among the droppable modules: it is append-only by protocol, not by policy, and no configuration may make it otherwise.

One setting that does not exist

Whether removals are recorded is not configurable. Every drop, cascade, supersession, demotion, prune, and redaction MUST be written to provenance, and no policy document, however constructed, may turn that off. Auditability is the property that makes exclusion trustworthy, so exposing it as a knob would make every other guarantee in this section conditional on a setting.

13 Divergence and Reconciliation

Section 7.4 requires an implementation to detect that two brains advanced from a common ancestor and to refuse to overwrite. Refusing is safe and incomplete: it leaves reconciliation to hand-editing, and it means a partial install can never be published back over the tag it came from. This section defines the reconciliation the refusal was standing in for.

13.1 One structural change

A snapshot names its predecessors in `parents`, a list (Section 6.5). That is the whole structural change: a field goes from scalar to list, and no new document is introduced. A linear history is the ordinary case and carries one entry; a root snapshot carries none; a reconciliation carries two or more.

Order is significant in exactly one way. The *first parent* is the history the reconciliation was performed onto, and every rule in this document that speaks of "the parent" means the first one: the scope a signature must hold (Section 8.4), the trust root in force (Section 8.5), and the difference a consumer reads as the change this snapshot made. A conforming implementation MUST treat the remaining parents as merged-in history and MUST NOT derive any authorization from them.

13.2 Why the structural part is nearly free

This is a genuine advantage over version control rather than a coincidence. Git merges lines of text; here what is merged is a set of immutable, content-addressed blocks. Nothing is ever "the same block, slightly modified": modifying a block yields a different identity (Section 6.1). A textual conflict is therefore not representable, and reconciliation of a single module is set arithmetic over identifiers, which converges regardless of the order the sides are combined in [37].

What makes it three-way rather than two-way is the common ancestor, and this is the part that cannot be skipped. Without it, a block present in one composition and absent from the other is ambiguous between "they added it" and "I dropped it", and those demand opposite outcomes. A conforming implementation MUST identify a common ancestor of the snapshots being reconciled, and MUST refuse with a distinguishable failure when the two histories share none (Table 15).

For each module, with B the ancestor's composition and X, Y the two compositions being reconciled, the result is

$$M = \big(B \cup X \cup Y\big) \setminus \big((B \setminus X) \cup (B \setminus Y)\big), \tag{2}$$

which is to say: everything either side added is kept, and everything either side removed stays removed.

Removal wins, and that is a feature. Equation 2 gives precedence to exclusion, so a block one side dropped does not return because the other side still held it. A related property follows from content addressing and is worth recording: dropped evidence cannot be smuggled back in by re-ingesting it. Re-registering the same source yields the same identifier (Section 5), so the reconciliation recognizes it as something this brain removed rather than as a new contribution, with no special rule required. Deliberately re-admitting a block that was dropped remains possible and remains an explicit commit, which is where a decision of that kind belongs.

The result is a candidate, not a commit. Equation 2 is applied per module and is individually correct in each, which is precisely why it is not sufficient. The invariants this architecture rests on run *between* modules, and a set operation never crosses that boundary. Section 13.4 is about what crosses it.

13.3 Merge, rebase, and squash are recording strategies

This inverts the intuition anyone brings from version control, and it is worth spelling out. In Git the three strategies differ both in how the result is computed and in how history is recorded: a rebase replays patches, and can therefore land on a tree a merge would not have produced. Here a snapshot is a complete statement of composition rather than a patch, so there is nothing to replay sequentially, and all three produce the same set of blocks. What differs is only the lineage recorded, and therefore who remains on record as the author (Table 19).

In Git, choosing among these is a question of tidiness. Once snapshots are signed (Section 8), it is a question of attribution: rebase and squash mint new snapshot identities, so a contributor's signature no longer covers anything in the resulting history and their work ends up signed by whoever performed the operation. That may be exactly what is wanted for a small reviewed contribution, but an implementation MUST report it as a consequence of the choice rather than let it happen silently. A conforming implementation MUST NOT present a rebased or squashed contribution as bearing the contributor's signature.

Table 19: The three strategies produce identical compositions and different histories. Once snapshots are signed, the difference is attribution rather than tidiness.

Strategy	Lineage recorded	Same blocks	Their snapshots kept	Their signature survives
Merge	Two or more parents.	Yes	Yes	Yes
Rebase	One parent: mine.	Yes	No	No
Squash	One parent, their snapshots collapsed into one.	Yes	No	No

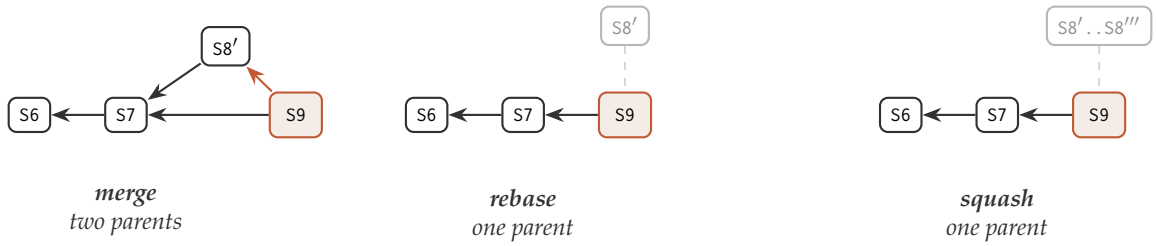

Figure 13: All three land the same blocks in `S9`. Only merge keeps the contributed snapshot in the history, and therefore only merge keeps the contributor's signature covering something. Greyed nodes are outside the resulting history.

Rebase and lineage rewrite are the same machinery. Replaying immutable blocks onto a new base is deterministic, but it changes snapshot identities, which invalidates signatures over them and any root already published. That is exactly the property of the lineage rewrite of Section 12, and unifying them is a simplification: both are legitimate only before publication, under version control's own rule about rewriting public history.

Squash collapses snapshots, not provenance. Squash is more useful here than in version control, because an ingestion session mints many intermediate snapshots nobody cares about individually. It MUST therefore be defined precisely or it breaks Principle 8: a squash compacts root history and MUST preserve every provenance record the collapsed snapshots produced. An implementation that discarded provenance while collapsing snapshots would be destroying the audit ledger to tidy a chain.

13.4 The conflicts that are real

Because no textual conflict is representable, the conflicts that remain are semantic, and there are few of them (Table 20). One deserves walking through, because it is the case that proves reconciliation cannot be purely structural.

Branch A drops canonical block *C*. Branch B adds a semantic block derived from *C*. Each module's set arithmetic is individually correct: the drop is respected in the canonical composition, the addition is respected in the semantic one. The result nonetheless violates R1, because a derived block now cites evidence that is not in the composition. The invariant that broke lives *between* modules, and Equation 2 never crosses that boundary. That single case is enough to establish that the reconciliation rule MUST be provenance-aware.

Table 20: Semantic conflict classes and how each is resolved.

Conflict	Resolution
One side drops canonical evidence the other side derived from.	Structurally invisible, semantically fatal. The derived block is rejected by the evidence check, exactly as the canonical cascade rejects a dependent (Section 12.3).
Both sides add blocks that contradict each other.	Not structurally detectable at all. Surfaces as `CONTRADICTED` when the result is re-validated, with the conflicting blocks named.
Both sides supersede the same block with different successors.	Both successors are admissible blocks and both supersession edges are recorded; the conflict is a precedence question and MUST yield `PENDING_REVIEW` rather than a silent winner.
The two sides carry different trust roots.	MUST NOT be reconciled automatically. Unioning two key lists grants the union of both sides' permissions, which defeats the quorum rule outright (Section 8.4). The reconciliation MUST be refused and resolved as an explicit governance act.

The rule. The structural reconciliation of Equation 2 is automatic, and its result MUST then be validated as if it were an ingestion (Section 10). Every question these conflicts raise, does this evidence exist, does this contradict what is already held, is this relation admissible, does the payload satisfy a registered schema, is already asked on the ingestion path, so no new machinery is introduced. Conflicts surface as candidates in `PENDING_REVIEW` or `CONTRADICTED` rather than as an unresolvable state. Put differently:

> A conflict in this protocol is a validation failure, not a differencing failure.

Only blocks that emerge `VALIDATED` enter the reconciled composition, and the reconciliation MUST NOT be committed while any candidate is still `PENDING_REVIEW`. The verdicts are the report a maintainer acts on, and they name which parts of a contribution fit before anything is decided.

13.5 Missing evidence has two causes, and the diagnosis matters

When a reconciled block cites evidence absent from the composition, the block is rejected either way: it cannot enter, for the same reason the canonical cascade does not repair its dependents. But *why* the evidence is absent determines what the contributor should be told, and provenance already holds the answer, because it is the removal ledger (Section 5).

A removal record exists. The evidence was dropped deliberately. The contribution rests on something this brain judged wrong, and resending it would re-import exactly what was excluded. The contributor SHOULD be told not to resend.

No record, and the identity is unknown. The evidence was never here: the contribution shipped a derived block without its canonical source. The work may be perfectly good and the transfer was incomplete. The contributor SHOULD be told to resend it whole.

Same verdict, opposite advice. A conforming implementation MUST distinguish the two, because collapsing them discards legitimate work over a packaging mistake and offers no way to tell which happened.

Rejection is not final. If a replacement canonical source is later registered, the `rederive` operation of Section 10 can rebuild what was lost against it. That is a separate, deliberate step taken afterwards, and it MUST NOT be folded into the reconciliation: a reconciliation that quietly re-derived rejected blocks against substituted evidence would be inventing knowledge in the middle of an operation the operator believes to be mechanical.

13.6 Contribution from outside

The deployment that motivates all of this is public read with restricted write (Section 8.12). What follows is how an external contribution works end to end (Figure 14). Nothing in it requires a new kind of object, and the first five steps require nothing this document had not already specified.

1. **The contributor extends the brain.** They install it at snapshot `S7`, no permission needed because the repository is public, ingest sources locally, and commit. Their snapshot names `S7` as its parent. They sign it with their own key, holding `propose`, and publish it to a repository they control. A contribution is therefore a signed snapshot, in another repository, whose parent belongs to the maintainer's history. The parent pointer is the entire link, exactly as a branch is in version control. There is no proposal object and no pending state.
2. **They notify the maintainer.** Out of band: an issue, a message, a URL. This protocol defines no notification mechanism, in the same way version control defines no pull requests. The snapshot is the protocol-level artifact; the workflow around it belongs to the forge.
3. **The maintainer fetches it.** Because everything is content-addressed, the transfer is only the delta: the contributor's brain shares every block reachable from `S7` with the maintainer's, byte for byte, so only genuinely new blocks and the new snapshot move. A contribution of forty blocks transfers forty blocks, not a brain.
4. **Nothing has changed yet.** The maintainer holds two histories locally while the published brain is untouched. `fetch` is defined to retrieve a remote history without moving the local pointer, which is the operation this step needs and the reason it is separate from `pull`.
5. **The contribution is judged mechanically, before any decision.** This is where the model departs usefully from version control. Reviewing a pull request means reading a diff; here the incoming blocks are candidates, and the validation of Section 10.3 applies unchanged. Every incoming block emerges with a verdict, so the maintainer learns which parts of a contribution fit before deciding anything, rather than inferring it by reading.
6. **The maintainer incorporates it** under one of the strategies of Section 13.3, re-validating the result as Section 13.4 requires, and signs the resulting snapshot with a key holding the scopes that snapshot's change requires.

Why `propose` is a scope and not a convention. A contributor's snapshot is a legitimately signed Boltzmann snapshot in every respect, which is the problem: without a scope distinguishing it, a consumer who pulled from the contributor's repository, or from a mirror that indexed it, would have no basis for treating it as anything other than the brain's current state. A key holding only `propose` produces snapshots that are attributable, verifiable, and explicitly not the published head. A conforming implementation MUST refuse to treat a `propose`-signed snapshot as a brain's current state unless a verification policy explicitly permits it (Section 8.9).

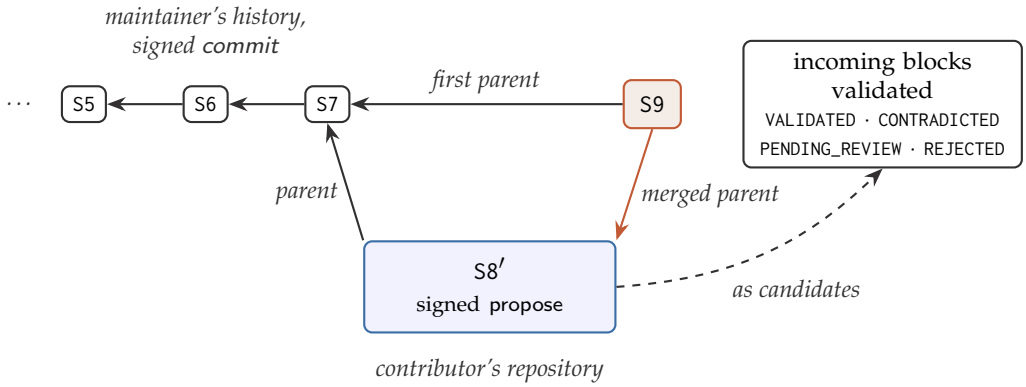

Figure 14: An external contribution needs no new object. The contributor's snapshot names a snapshot of the maintainer's history as its parent, which is the whole link; the maintainer fetches only the delta, validates the incoming blocks as candidates, and records the outcome as a reconciliation whose first parent is their own history.

13.7 Publishing back from a partial install

Section 7.4 states that a partial install cannot be published back over the tag it came from, because doing so would drop the modules that were never fetched. Reconciliation resolves this rather than working around it.

A snapshot produced by a partial install names roots only for the modules that install holds. Publishing it back MUST be expressed as a reconciliation with the remote head, in which the modules the publisher does not hold take their roots from the remote unchanged, and the modules it does hold are reconciled by Equation 2. The published snapshot then names every module the remote named, no module regresses, and the `source-snapshot` annotation of Section 7 continues to record the full snapshot a projection was taken from. A conforming implementation MUST NOT publish a partial snapshot over a tag naming modules that snapshot omits.

14 Knowledge Lifecycle

The full lifecycle ties the previous sections together:

1. **Source:** the user provides information via their LLM and a skill.
2. **Canonical registration:** the protocol preserves the original, computes its hash, and records provenance.
3. **Delegation:** the protocol returns a task and a processing schema.
4. **Interpretation:** the LLM proposes episodic, semantic, and procedural blocks and relations.
5. **Validation:** the protocol checks structure, references, duplicates, and policies.
6. **Commit:** accepted candidates receive hashes and are stored as immutable blocks.
7. **Indexing:** B^+ trees, lexical search, vectors, and graph are updated.
8. **Versioning:** the Merkle roots of the affected modules change.
9. **Publication:** OCI distributes only the modified physical modules.

10. **Attribution:** the published snapshot is signed, and a consumer verifies integrity and authorship as two separate results (Section 8).
11. **Query:** the LLM obtains Evidence Bundles and transforms them for the user.
12. **Contribution:** an outside party extends the brain in a repository of their own, and the maintainer fetches, validates, and reconciles the result (Section 13).
13. **Reconsolidation:** new evidence or corrections generate later candidates and snapshots.
14. **Forgetting and retention:** non-episodic modules drop wrong or obsolete blocks, rebuild their Merkle roots, and cascade through provenance (canonical drops always cascade); prune what no retained root needs; episodic memory supersedes; redaction destroys retained bytes only when policy requires it (Section 12).

The canonical, episodic, semantic, procedural, and provenance memories remain at the center of the architecture. The change consists solely in decoupling cognitive processing: the protocol organizes and governs; the external LLM interprets and contextualizes.

15 Security Considerations

A document claiming to be a standard owes a plain statement of what it guarantees, what it does not, and where the burden sits. Much of the burden here lands on the consumer, and saying so is the point rather than an admission. Table 21 summarizes; the subsections that follow treat the cases where the architecture itself creates the exposure.

Table 21: What the protocol guarantees against each capability, and what it does not.

An adversary who can	Is prevented from	Is not prevented from
Modify bytes in transit or at rest	Passing any verification: every identifier is recomputed from the bytes (Section 8.1).	Denying service by corrupting what a consumer needs.
Publish a fabricated brain under someone else's name	Being accepted by any consumer holding a pin (Section 8.11).	Being accepted on first contact by a consumer with no pin (Section 8.7).
Control the registry	Fabricating state: signatures and roots verify offline.	Withholding newer snapshots and serving a stale signed one (Section 15.3).
Steal one authorized private key	Revising the trust root, when the quorum exceeds one (Section 8.4).	Signing snapshots within that key's scopes until a compromise record is published (Section 8.5).
Get content into an ingested source	Having it committed without a verdict, or without provenance naming its producer (Section 10.3).	Having it faithfully preserved, retrieved, and replayed into a model (Section 15.1).
Read a published brain	Learning anything the publisher did not put in it.	Recovering approximations of dropped content from a stale travelling index (Section 6.6).

15.1 Untrusted content reaching a model is structural, not incidental

This architecture routes untrusted content into a language model by design. Canonical evidence is preserved verbatim because Principle 2 requires it, retrieved because that is what the query path is for, and handed to an external model that acts on it. Indirect prompt injection [12] is therefore a structural property of the design rather than an integration detail: a poisoned source that survives ingestion is replayed into every model that later queries the brain, and it arrives carrying the protocol's own verification stamp attesting that it is authentic evidence. Retrieval-augmented systems have been shown to be corruptible by a very small number of injected passages [44], and content addressing does nothing to prevent it: a poisoned block is faithfully preserved, correctly identified, and honestly attributed.

The consequence must be stated precisely, because the alternative is a false sense of safety:

What `verified` means, and what it does not

The `verified` field of an Evidence Bundle asserts that a block is authentically the knowledge that was committed to this brain, at this snapshot, by an identified key. It asserts nothing whatsoever about whether the content is true, or safe to act on. A conforming implementation MUST NOT describe verification as sanitization, and MUST NOT present canonical evidence to a model as trusted instruction.

Where the burden sits follows from the boundary of Section 9. The protocol governs what is stored and returns data rather than prose; the caller decides what to do with it. A caller that hands retrieved evidence to a model SHOULD treat it as untrusted input, separate it from instructions, and constrain what the model is permitted to do as a result of reading it. The protocol contributes what it can contribute honestly: every retrieved block names its canonical source, so a suspect claim can be traced to the bytes that produced it, and every derived block names its producer, so a poisoned batch can be excluded as a set.

15.2 Poisoning in bulk, and its remedy

Because provenance records the producer of every derived block at the granularity of a model, pipeline, ingestion batch, or actor, `drop_by_producer` (Section 12.3) can exclude everything a compromised producer made in one operation. This is remediation and not prevention, and it should be described as such: it shortens the window during which poisoned knowledge is served, and it does not stop the poisoning. Its value is that the window is bounded by a query over provenance rather than by an audit of a corpus.

Signatures narrow the window further in one specific way. Because a snapshot's author is authenticated, a poisoned commit is attributable to a key rather than to a declared name, and `compromised_from` withdraws every snapshot that key signed from a chosen chain position (Section 8.5). Attribution is the precondition for that response, which is the operational argument for Section 8 beyond impersonation.

15.3 Freshness: the limitation signing does not remove

Everything verifies offline against a pinned trust root, so a hostile or merely stale registry cannot fabricate state. It can still *withhold*: serve an older, legitimately signed snapshot indefinitely, while concealing that newer ones exist. Every signature verifies, every root recomputes, the trust root matches the pin, and the consumer is nonetheless reading a brain that was current months ago, possibly one from before a drop that removed knowledge for legal reasons.

Nothing in this version detects that, and a consumer MUST NOT infer freshness from a successful verification. The two known answers are expiring metadata, where a signed statement asserts a validity window and its absence is itself a signal, as in The Update Framework's

timestamp role [36], and an append-only transparency log, where a consumer can check that the snapshot it was served is the latest one the log has witnessed [17]. Both require a party outside the artifact, which is why neither is specified here: the offline property of Section 8.1 is the one thing this design is unwilling to trade, and both answers weaken it for consumers that cannot reach the extra party. Section 17 states this as open.

15.4 Erasure does not propagate

The limitation of Section 12.5 is worth restating in security terms. A redaction destroys bytes in the brain that performs it. It does not reach copies already pulled elsewhere. The protocol can publish the fact of a removal, and a conforming implementation must report which blocks are tombstoned, but a distributed brain can only signal destruction, not guarantee it. Any deployment whose obligations require guaranteed destruction must not distribute the content whose destruction it may later have to guarantee, and no protocol feature substitutes for that decision.

15.5 Hash agility

Every digest in this version is SHA-256, and every digest is written with its algorithm as a prefix, so a future version can introduce another without ambiguity. What such a transition [2] cannot be is an in-place upgrade. A `block_id` is knowledge identity, and a Merkle root is version identity, so recomputing them under a different algorithm produces a different brain: every identifier changes, structural sharing with existing copies is lost, and signatures over old snapshots continue to cover the old values only. A migration is therefore a re-identification of a whole brain, carried out deliberately and recorded in provenance, and a conforming implementation MUST NOT mix algorithms within one snapshot or treat identifiers under different algorithms as comparable.

15.6 What a consumer is responsible for

The properties in this document hold for a consumer that does the following, and an implementation SHOULD make each of them the default rather than an option:

- Recompute integrity; never accept a publisher's assertion of it (Section 8.9).
- Pin a trust root on first contact, and refuse a change that does not satisfy the quorum rule (Section 8.7).
- Report integrity and authenticity separately, and never collapse "intact" into "authentic" (Section 8.9).
- Treat retrieved evidence as untrusted input to a model (Section 15.1).
- Treat a successful verification as saying nothing about freshness (Section 15.3).

16 Conformance

A protocol whose conformance is asserted rather than demonstrated is a convention, not a standard. This section states what an implementation must demonstrate, and how.

16.1 Roles

Conformance is claimed per role, not for the protocol as a whole (Table 14). An implementation MAY claim the **reader** role alone, which is the common case for a consumer that installs a published brain and queries it. Claiming **authenticity**, **writer**, **retention**, or **distribution** requires the reader role as well, since each of them is defined in terms of reading a snapshot before changing it or attesting to it. An implementation that claims all five is *fully conforming*.

Authenticity is claimable separately for a reason that is easy to mistake for a weakness. A consumer that recomputes integrity while holding no trust anchor is not a degraded client: it is the zero-configuration case Section 8.1 guarantees, and requiring signature verification for a reader to be conforming would make offline integrity conditional on configuration the protocol promises it does not need. What no implementation may do is claim an authenticity it did not check (Section 8.9).

Partial claims are not a weakness to be tolerated but the mechanism that makes selective installation meaningful: a client that only reads never needs validation, contradiction detection, or a packing format, and requiring it to implement them to be conforming would push implementers toward claiming nothing at all.

16.2 Golden vectors

Agreement on the identity layer cannot be established by prose, because every disagreement in it is silent: two implementations that serialize differently do not fail, they simply compute different identifiers for the same knowledge and stop sharing blocks. Conformance is therefore anchored in published test vectors, covering the four places where a divergence would be invisible:

- **Serialization**: values and the exact canonical bytes they produce, including the cases that MUST be rejected (Section 6.2).
- **Block identifiers**: envelopes and the `block_id` each one hashes to.
- **Merkle roots**: leaf sets and the root the construction of Section 6.3 produces, including the empty set and odd-sized trees, where a wrong split is otherwise easy to miss.
- **Inclusion proofs**: proofs and the roots they must reconstruct.
- **Signatures**: snapshots, published test key pairs, namespaces, and the verdict a verifier MUST reach, including a signature valid under the wrong namespace, a key authorized at one chain position and retired at another, a key with a `compromised_from` record, and a trust-root revision that fails the quorum rule (Section 8).
- **Reconciliation**: an ancestor composition and two branch compositions, with the set Equation 2 must produce, including the case where one side dropped what the other retained (Section 13.2).

These vectors MUST be plain data, readable without executing any implementation of this protocol. An implementation in another language demonstrates agreement by reproducing them, which is a stronger claim than passing a test suite supplied by a reference implementation, because it requires no dependency on that implementation at all.

Behavioral requirements that cannot be reduced to fixed vectors, such as the guarantee that a query returns only verified blocks, or that a drop yields a composition excluding the dropped identity, are stated as the MUST clauses of the sections that define them. A reference implementation MAY publish an executable suite for these; conformance is defined by the clauses, not by the suite.

17 Conclusion and Future Work

We presented Boltzmann Brain, a conceptual architecture for portable, verifiable, and model-agnostic knowledge. Its core commitment is a clean separation of concerns: the brain conserves, validates, and retrieves, while an external, interchangeable LLM interprets and contextualizes. From this commitment follow the five-module memory model, the content-addressed blocks versioned by Merkle DAGs, the derived indices, the three levels of hashes, a distribution model in which OCI Artifacts enable selective installation and incremental updates, and an account of how canonical evidence roots re-derivation, how non-episodic modules drop wrong knowledge with a privileged cascade when that knowledge's source is excluded, rebuild Merkle roots, and reclaim unreachable blocks, with redaction reserved for destroying retained bytes.

Beyond the architecture, this version fixes the interoperability surface: the block envelope and its canonical serialization, the Merkle construction and its layout identifier, the composition and snapshot documents, the schema-versioning rules, the distribution encoding, the ingestion and query contracts, and a conformance kit anchored in golden vectors.

It then addresses the two gaps an earlier version named as open, and the treatment of each turned out to be shorter than expected because the existing structure already did most of the work.

Authenticity was a question of where, not of how much. Every guarantee the hash structure makes is a guarantee of integrity, and integrity is already total: any consumer, offline and unconfigured, can confirm that a brain is exactly the brain its identifiers describe. What no amount of hashing can supply is authorship, because internal consistency is cheap to manufacture. Section 8 adds the one assertion the hashes cannot make, and adds it in one place: a snapshot already commits to every module root and, through its parents, to the entire prior history, so a single detached signature over it covers everything and nothing else needs signing. The keys authorized to produce such a signature travel inside the brain so that verification stays offline, their scopes are the protocol's own operations rather than invented roles, revising the list requires a quorum of keys authorized in the previous revision, and validity is decided by position in the hash-linked chain rather than by a timestamp a signer could forge. The one thing that cannot come from inside is trust at first contact, and that limitation is stated plainly.

Divergence was a question of what to record, not of what to compute. Section 13 makes `parents` a list, which is the whole structural change. Because a module composition is a set of immutable, content-addressed blocks, no textual conflict is representable and the structural reconciliation is set arithmetic; because a snapshot is a complete statement of composition rather than a patch, merge, rebase, and squash all produce the same blocks and differ only in the lineage they record, which once snapshots are signed turns the choice into a question of attribution. The conflicts that remain are semantic, and they are resolved by re-validating the reconciled result as if it were an ingestion, so a conflict surfaces as a validation verdict instead of an unresolvable state. A contribution from outside then needs no new object at all: it is a signed snapshot, in another repository, whose parent belongs to the maintainer's history.

Section 15 states the adversarial model both of those imply, including the exposure the architecture creates by design: preserved evidence is routed into an external model, so a poisoned source that survives ingestion is replayed with the protocol's own verification stamp attached. Verification means that content is authentically what was committed, never that it is true or safe to act on, and this document says so where a reader will meet it.

What the protocol still owes. Six questions remain, and they are smaller and better specified than the two just closed.

- **Freshness.** Everything verifies offline, so a hostile registry cannot fabricate state, but it can withhold: serve an older, legitimately signed snapshot indefinitely. Signing does not fix this. The known answers, expiring metadata [36] and an append-only transparency log [17], both require a party outside the artifact, which is why neither is adopted here without deciding what to trade for it. This is the largest of the six.
- **Removal from a travelling index.** The protocol now requires an index to be bound to its composition, and requires a stale one to be omitted or reported rather than shipped. It does not define deletion of a single entry, because deletion from an approximate nearest-neighbour structure is engine-specific and lossy. Whether a protocol-level removal is worth specifying, given that omission is always available, is open.
- **A deterministic path for partial evidence loss.** A derived block that loses one of several sources cannot survive, because its citations are inside its identity. Rebuilding it currently costs a full round trip through the external model even though the surviving evidence has not changed, and a cheaper deterministic path for that specific case is undefined.
- **Propagation of erasure.** A redaction destroys bytes locally. The protocol can publish the fact of a removal, but a distributed brain can only signal destruction, not guarantee it across copies already pulled.
- **Decay in episodic memory.** Which function governs demotion is left to an implementation, and it is not obvious that it should be.
- **Federation across trust boundaries.** Reconciliation assumes one governed history. Nothing yet says how two independently governed brains, with different trust roots, cite each other's evidence without one of them adopting the other's authorities.

Validating all of this against real, multi-source brains remains the natural next step, and the properties added here are the ones that make such a validation meaningful beyond a single author: a brain that several parties extend is exactly the case where integrity was never the hard part.

References

[1] Anthropic. Model context protocol: Architecture overview. Model Context Protocol Specification, 2024. https://modelcontextprotocol.io.

[2] Elaine Barker and Allen Roginsky. Transitioning the use of cryptographic algorithms and key lengths. Technical Report NIST SP 800-131A Rev. 2, National Institute of Standards and Technology, 2019. https://doi.org/10.6028/NIST.SP.800-131Ar2.

[3] Robert A. Bjork and Elizabeth L. Bjork. A new theory of disuse and an old theory of stimulus fluctuation. In *From Learning Processes to Cognitive Processes: Essays in Honor of William K. Estes*, volume 2, pages 35–67. Erlbaum, 1992.

[4] Scott Bradner. Key words for use in RFCs to indicate requirement levels. Technical Report RFC 2119, Internet Engineering Task Force, 1997. https://www.rfc-editor.org/rfc/rfc2119.

[5] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2 edition, 2014. Chapter 10, Git Internals.

[6] Cognee. The cognify operation. Cognee Documentation, 2025. https://docs.cognee.ai.

[7] Cognee. Core concepts overview. Cognee Documentation, 2025. https://docs.cognee.ai.

[8] Ronald L. Davis and Yi Zhong. The biology of forgetting: A perspective. *Neuron*, 95(3): 490–503, 2017.

[9] European Parliament and Council of the European Union. Regulation (eu) 2016/679 (general data protection regulation), article 17: Right to erasure. Official Journal of the European Union, L 119, 2016. `https://eur-lex.europa.eu/eli/reg/2016/679/oj`.

[10] Travis D. Goode, Kazumasa Z. Tanaka, Amar Sahay, and Thomas J. McHugh. The hippocampal engram as a memory index. *Neurobiology of Learning and Memory*, 2018.

[11] Google Cloud. Introducing the open knowledge format. Google Cloud Blog, 2026. `https://cloud.google.com/blog`.

[12] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.

[13] Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (EdDSA). Technical Report RFC 8032, Internet Engineering Task Force, 2017. `https://www.rfc-editor.org/rfc/rfc8032`.

[14] Richard Kissel, Andrew Regenscheid, Matthew Scholl, and Kevin Stine. Guidelines for media sanitization. Technical Report NIST Special Publication 800-88 Revision 1, National Institute of Standards and Technology, 2014. Defines cryptographic erase.

[15] James J. Knierim and Joshua P. Neunuebel. Tracking the flow of hippocampal computation: Pattern separation, pattern completion, and attractor dynamics. *Neurobiology of Learning and Memory*, 129:38–49, 2016.

[16] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.

[17] Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. Technical Report RFC 9162, Internet Engineering Task Force, 2021. Defines the Merkle Tree Hash; obsoletes RFC 6962. `https://www.rfc-editor.org/rfc/rfc9162`.

[18] Microsoft. Graphrag: Indexing architecture and methods. Microsoft GraphRAG Documentation, 2024. `https://microsoft.github.io/graphrag`.

[19] Microsoft. Graphrag: Query engine overview. Microsoft GraphRAG Documentation, 2024. `https://microsoft.github.io/graphrag`.

[20] Alistair Miles and Sean Bechhofer. SKOS simple knowledge organization system reference. W3C Recommendation, 2009. Concept schemes with `broader` and `narrower` relations. `https://www.w3.org/TR/skos-reference/`.

[21] MITRE. CVE-2012-2459: Merkle tree second-preimage ambiguity. Common Vulnerabilities and Exposures, 2012. Duplicating the last node on an odd level lets two distinct leaf sets produce the same root. `https://www.cve.org/CVERecord?id=CVE-2012-2459`.

[22] John X. Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M. Rush. Text embeddings reveal (almost) as much as text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12448–12460, 2023.

[23] Karim Nader and Oliver Hardt. A single standard for memory: The case for reconsolidation. *Nature Reviews Neuroscience*, 10(3):224–234, 2009.

[24] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2353–2367, 2022. Signatures published as separate objects referring to the artifact they cover.

[25] Open Container Initiative. Oci descriptor specification. OCI Specifications, 2024. `https://github.com/opencontainers/image-spec`.

[26] Open Container Initiative. Oci image manifest specification. OCI Specifications, 2024. `https://github.com/opencontainers/image-spec`.

[27] Open Container Initiative. OCI distribution specification: Listing referrers. Specification, 2024. `https://github.com/opencontainers/distribution-spec`.

[28] OpenSSH. PROTOCOL.sshsig: Signing and verifying data with SSH keys. OpenSSH protocol documentation, 2019. Detached signatures over arbitrary data under a caller-chosen namespace. `https://github.com/openssh/openssh-portable/blob/master/PROTOCOL.sshsig`.

[29] ORAS. Understanding oci artifacts. ORAS Documentation, 2024. `https://oras.land`.

[30] Rosa C. Paolicelli, Giulia Bolasco, Francesca Pagani, Laura Maggi, Maria Scianni, Patrizia Panzanelli, Maurizio Giustetto, Tiago Alves Ferreira, Eva Guiducci, Laura Dumas, Davide Ragozzino, and Cornelius T. Gross. Synaptic pruning by microglia is necessary for normal brain development. *Science*, 333(6048):1456–1458, 2011.

[31] S. R. Ranganathan. *Prolegomena to Library Classification*. Asia Publishing House, Bombay, 3rd edition, 1967. Faceted classification: one hierarchy cannot serve every access need, so a document takes coordinates on several independent axes.

[32] Nelson Rebola, Mario Carta, and Christophe Mulle. Operation and plasticity of hippocampal ca3 circuits: Implications for memory encoding. *Nature Reviews Neuroscience*, 18(4): 208–220, 2017.

[33] Roger L. Redondo and Richard G. M. Morris. Making memories last: The synaptic tagging and capture hypothesis. *Nature Reviews Neuroscience*, 12(1):17–30, 2011.

[34] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON canonicalization scheme (JCS). Technical Report RFC 8785, Internet Engineering Task Force, 2020. `https://www.rfc-editor.org/rfc/rfc8785`.

[35] Tomas J. Ryan and Paul W. Frankland. Forgetting as a form of adaptive engram cell plasticity. *Nature Reviews Neuroscience*, 23(3):173–186, 2022.

[36] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Survivable key compromise in software update systems. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*, pages 61–72, 2010. The Update Framework: threshold-signed root metadata, role separation, and expiring timestamps.

[37] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 386–400, 2011. Convergence of set-valued replicas under commutative merge.

[38] Garry Tan. Brain vs memory vs session. GBrain, GitHub repository, 2026. https://github.com/garrytan/gbrain.

[39] Garry Tan. Pluggable engine architecture. GBrain, GitHub repository, 2026. https://github.com/garrytan/gbrain.

[40] Garry Tan. Two-repo architecture: Agent behavior vs world knowledge. GBrain, GitHub repository, 2026. https://github.com/garrytan/gbrain.

[41] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bytes and bytes. In *28th USENIX Security Symposium*, pages 1393–1410, 2019.

[42] Michael A. Yassa and Craig E. L. Stark. Pattern separation in the hippocampus. *Trends in Neurosciences*, 34(10):515–525, 2011.

[43] Tatu Ylonen and Chris Lonvick. The secure shell (SSH) protocol architecture. Technical Report RFC 4251, Internet Engineering Task Force, 2006. Section 4.1 states the first-contact key problem and the leap-of-faith model this document adopts for bootstrap. https://www.rfc-editor.org/rfc/rfc4251.

[44] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. arXiv:2402.07867, 2024. https://arxiv.org/abs/2402.07867.